Technische Universität Berlin

Master's Thesis

Electrical Engineering and Computer Science
Institute of Software Engineering and Theoretical Computer Science
Neural Information Processing Group

Learning Neural Network Structure that Aids the Interpretability of the Underlying Processes

Aleksandr Chebykin
Matriculation Number: 406154

1. Reviewer	Prof. Dr. Klaus Obermayer Neural Information Processing Group Technische Universität Berlin
2. Reviewer	Prof. Dr. Manfred Opper Methods of Artificial Intelligence Technische Universität Berlin
Supervisor	Prof. Dr. Klaus Obermayer
Co-Supervisors	Heiner Moritz Spieß Youssef Kashef

December 21, 2020

Declaration

I hereby declare that the thesis submitted is my own, unaided work, completed without any unpermitted external help. Only the sources and resources listed were used.

The independent and unaided completion of the thesis is affirmed by affidavit:

Berlin, December 21, 2020

Aleksandr Chebykin

Abstract

Neural network interpretability is an important research direction that serves as a supplement to the more performance-oriented landscape of modern deep learning research. Driven both by human curiosity and by legal requirements, steps are being taken to rid neural networks of their "black box" status.

Investigating network behaviour is not a simple task. It is hindered by the question of the proper level of abstraction, by results that are hard to quantify, and by the vast size of the networks.

This thesis introduces a potential solution to the last problem in a form of an algorithm that can learn sparse network structure without suffering a big performance loss. This is achieved by pruning individual kernels in a convolutional neural network, instead of whole neurons or individual weights in a kernel, as is common practice. Resulting sparse networks can be reasonably easily examined by a human researcher. In the experiments a network is trained on the multi-task CelebA dataset.

Can the resulting network serve as a good model of the dense networks? After examination of the sparse network, the most interesting observation — minimal activations encode meaningful information — is further tested in dense networks and in both multi-task and single-task settings. Evidence indicates that the phenomenon is general, which suggests that the proposed approach of deriving insights from sparse models serves its goal.

Zusammenfassung

Die Interpretierbarkeit neuronaler Netze ist eine wichtige Forschungsrichtung, welche als eine Ergänzung zur leistungsorientierten Landschaft der modernen Deep-Learning Forschung dient. Angetrieben sowohl von menschlicher Neugier als auch von gesetzlichen Anforderungen werden Schritte unternommen, um neuronale Netze von ihrem "Black Box"-Status zu befreien.

Die Untersuchung des Netzwerkverhaltens ist keine einfache Aufgabe. Es wird durch die Frage nach der korrekten Abstraktionsebene, durch schwer quantifizierbare Ergebnisse und durch die immense Anzahl an Parametern innerhalb der Netzwerke erschwert.

Diese Arbeit stellt eine mögliche Lösung für das letzte Problem in Form eines Algorithmus vor, der eine spärliche Netzwerkstruktur lernen kann, ohne einen großen Leistungsverlust zu erleiden. Dies wird erreicht, indem einzelne Kernel in einem Convolutional Neural Network entfernt werden, anstatt wie üblich ganze Neuronen oder einzelne Gewichte in einem Kernel. Die resultierenden spärlichen Netzwerke können von einem menschlichen Forscher relativ einfach untersucht werden. In den Experimenten wird ein Netzwerk auf dem Multi-Task CelebA Datensatz trainiert und evaluiert.

Kann das resultierende Netzwerk als gutes Modell für die dichten Netzwerke dienen? Nach der Untersuchung des spärlichen Netzwerks wird die interessanteste Beobachtung, dass minimale Aktivierungen aussagekräftige Informationen codieren, in dichten Netzwerken und sowohl in Multi-Task- als auch im Single-Task-Lernen weiter getestet. Die Ergebnisse dienen als Beweise dafür, dass das Phänomen allgemein ist. Damit zeigt sich, dass der vorgeschlagene Ansatz, Erkenntnisse aus spärlichen Modellen abzuleiten, seinen Zweck erfüllt.

Contents

1 Introduction

Modern Convolutional Neural Networks (CNNs) have achieved remarkable results in various computer vision problems, such as classification [65], detection [56], and segmentation [58]. Despite these successes, only a modest amount of headway has been made in understanding what CNNs actually learn [20] and making them understandable to a human observer [69]. The status of "black boxes" limits usefullness of neural networks in many contexts, including medicine [54] and banking [3], because important decisions that bear real-world consequences must be understood by experts, as well as explained to affected persons [14].

Thus it comes as no surprise that there exists a significant research interest in the area of interpretability and explainability of neural networks [4, 5, 44] (for a detailed description of related work see section 2). Since the overarching goal is to clearly understand processes happening within modern networks of significant complexity and size, much of research is performed directly on these networks. However, this approach is problematic due to the overwhelming size of the networks: thousands of neurons and millions of connections cannot be individually examined by a human researcher in a reasonable timeframe. This leads to concentration on a subset of models that are not overwhelmingly big, yet not very modern, such as AlexNet [29, 72]. Alternatively, simple models of plain sequential architecture can be used [24, 30], but they come with their own downsides: size that is still not human-scale and worse performance. It is unclear whether conclusions drawn from worse-performing models transfer to better-performing ones. This comes in addition to the same concern of conclusion transferability from models of plain architecture to models of complicated architecture.

Ideal networks to examine would be those that are both human-scale and similar enough to the modern ones so that the gained insights would transfer. The main goal of this thesis is to come up with a way to construct such networks.

Our approach (presented in section 3) can be viewed as an online pruning algorithm. It allows us to construct networks that are extremely sparse (up to only 0.1% connections remaining) yet still perform similarly to dense versions. The algorithm is architecture-independent, we apply it on the ResNet-18 [18] architecture on the CelebA [34] dataset. The resulting sparse network can be readily interpreted

by a human, as most neurons and connections in it fit on a single screen, which helps with visual inspection. Individual connections are analyzed and often found interpretable. Their interpretations are corroborated by causal evidence, such as connection removal (which in sparse networks leads to visible behavioural changes) and modifying individual attributes of input images with Generative Adversarial Networks. The latter, however, turns out to be problematic due to side-effects produced by GANs.

The most intriguing effect found in the sparse network is the apparent presence of meaning in near-zero activations. This is surprising because ReLU is used as non-linearity, and the common interpretation of a neuron with this activation function is that the neuron activates highly for a specific feature and lowly for everything else. This interpretation does not allow small activations to correspond to a specific narrow feature, and yet that is what we find.

In order to see whether the effect is general (rather than specific to the sparse models trained by the proposed algorithm), in section 4 it is further tested in a dense CelebA-trained network, and then dense networks trained on single-task datasets (CIFAR-10 [28] and ImageNet [12]). The results are indeed consistent between sparse and dense networks. The phenomenon is examined further with feature visualization techniques; its impact on network behaviour is quantitatively measured.

The final discussion of the results, their weaknesses, and potential future work is presented in section 5.

2 Related Work

2.1 Learning network structure

The most general approach to learning network structure is Neural Architecture Search [74]. It allows combining basic building blocks like convolution operations or various activation functions into a computational graph. A whole network can be learned this way [74], though usually due to computational constraints only several blocks that are each several operations deep are learned, and then stacked many times to construct a deep network [48, 50]. Since the choices of which operations to use and how to connect them are discrete, they cannot be simply learned via backpropagation. Instead, various approaches for optimizing them are used, including evolutionary algorithms [50], reinforcement learning [48, 74], Gumbel-Softmax relaxation [25] of categorical distributions [53].

Less general approaches to structure learning introduce constraints on what can be learned and what cannot.

MaskConnect [1] learns to connect ResNet blocks to previous blocks via skip connections; when applied to ResNeXt architecture [66], the approach learns connections between branches. Connections are represented as binary masks, for learning they are parameterized with real values. Each node, for which input connections are learned, is assumed to take K inputs (K is a hyperparameter). To pick currently active connections, the real-valued mask parameters are normalized to represent multinomial probability distribution, which is then sampled K times without replacement.

Branched Multi-task Networks [59] is an interesting approach for learning which parameters to share between which tasks in a multi-task setting. It allows learning network structure in form of a tree by using Representation Similarity Analysis [27] to pick good depths for branching based on the measured similarity of task representations in single-task networks for each task.

Another approach for learning structure in the multi-task setting is "Stochastic Filter Groups" [9]: the method learns to assign each filter to a task-specific group or to a group that is shared across the tasks. Filters in a task-specific group are connected

only to the filters in the next layer that also are specific for that task; filters in the shared group are connected to all filters in the next layer. Connectivities are parameterized with real values: each filter has a corresponding vector of probabilities to be assigned to a specific group; the probabilities are optimized via variational inference.

Connectivity structure need not be fixed after learning: Soft Ordering of Shared Layers [38] is a method that learns dynamic paths through layers (different for every task in the multi-task setting). Interestingly, every layer in the network consists of the same parameters (that are divided into several parallel parts), and it is the connections between parts that mostly define network behaviour. Connections are represented with real values that are passed through softmax before being applied to the activations.

Possibly the most restrictive approach to structure learning is learning group convolutions. Group convolution [29, 66] is a modification of ordinary convolution where not every input is connected to every output, but rather the connectivity matrix has structured sparsity, which is valuable because of the efficiency it brings. CondenseNet [21] prunes weakest connections based on the L_1 norm of the corresponding weight in several stages; filters are separated into groups before optimization, during which Group Lasso [68] regularization is applied so that all filters in a group would be encouraged to use the same inputs.

Fully Learnable Group Convolution [62] represents connectivity with two real-valued matrices: the first matrix is used to assign each input filter to a group, the second matrix is used to assign each output filter to a group. To binarize a matrix, values for different groups are compared for each filter, connection corresponding to a group with the biggest value is set to 1, all the other connections are set to 0. The two resulting binary matrices can be multiplied to get the connectivity that is applied to the weight of the convolution by element-wise multiplication (application of connectivity via element-wise multiplication is very efficient, we also use this approach in section 3.2).

Groupable Convolutional Neural Network [71] suggests that connectivity can be represented via a low-dimensional matrix that can be upsampled using Kroenecker product to get connectivity of the desired shape. The approach introduces very few additional parameters, at the cost of restricting possible connectivities that can be learned. The parameters are again real-valued, to binarize them, their sign is evaluated. The values are updated via a straight-through estimator [17] that treats binarization as identity function and passes gradients through it without change.

Our approach is different from learnable group convolutions in that we are interested in learning connections on the level of individual neurons rather than groups of them. Additionally, we are interested in understanding the network rather than making it efficient.

The idea that network structure can be learned in order to aid interpretation has already seen some development. For example, in [70] an explanatory graph is learned for a given CNN, with different nodes in the graph corresponding to different object parts, and connections between layers corresponding to part hierarchy. Unlike this work, we learn connectivity of the network itself and do not restrict it to object parts.

Pruning of weights can be viewed as a special kind of structure learning where the only operation available is removal of a part of a network. Generally, there are two kinds of pruning: unstructured — where individual weights are removed — and structured — where individual neurons are removed together with all the weights connected to them. Unstructured pruning is more flexible and therefore allows for better compression at the same performance level when compared to the structured pruning [15]. However, contemporary hardware does not allow to take advantage of unstructurally sparse matrices: for practical performance that means that unstructurally pruned networks are not much faster than their dense versions. Structured pruning, on the other hand, leads to measurable efficiency gains, which makes it useful in practice despite worse compression levels [67].

Notable structured pruning algorithms include pruning filters with the smallest L_1 norm of weights [31], pruning filters by training an additional layer that outputs whether a filter should be kept or not via sigmoid that is annealed during training to output binary values [35], pruning filters with smallest scaling factors of the consequent Batch Normalization layer [33]. The unstructured approaches are less diverse, as they always prune based on the magnitude of the individual weights, e.g. [73].

Our approach (described in section 3) falls between these categories because we pursue objectives different from space or performance efficiency, namely: ease of understanding by a human. So we neither remove individual weights of convolutional filters (as unstructured pruning would do) nor individual neurons as a whole (as structured pruning would do), instead we remove individual connections between neurons, which for CNNs means removing 2D kernels. Sometimes this is referred to as kernel-wise pruning [2].

2.2 Examining network behaviour

There is a wide variety of different approaches to the interpretability of neural networks. Usually, a distinction is made between local methods that aim to explain network behaviour on a single input example (usually by constructing something similar to a saliency map, with representative algorithms including Layer-wise Relevance Propagation [4, 8], integrated gradients [55], Grad-CAM [51]) and global methods that search for explanations not limited to a single input (for example, learning to decompose activation tensors into understandable neuron groups [46] or tracking linear projections of networks' representations during training [32]). We are interested in the specific sub-category of global explanations that deals with individual neurons.

Most often, this translates to inspecting maximal activations. The most intuitive approach is selecting dataset examples that activate a neuron the most. However, this approach can be misleading because it may include patches that correlate with high activations rather than cause them [45]. To combat this, techniques like Activation Maximization [13] were introduced. Activation Maximization consists of synthesizing maximally excitatory inputs for a given neuron by gradient descent. An advantage of this method is that it starts from noise and therefore any visible features in the final image appear there because they directly cause a neuron to fire more (example images produced by the technique can be seen in Fig. 3.3). However, to get better images, various regularization approaches are used, which nudge optimization to produce images that have specific qualities (e.g. more low frequencies) — this weakens the argument that every pixel takes the value it takes because that is what excites the neuron the most.

Another established way of interpreting individual neurons is Network dissection [5]. The approach consists in interpreting spatial activations of a neuron as a binary segmentation mask, which is done by thresholding activations by a value that is bigger than 99.5% of the neuron's activations over the whole training set. These segmentation masks are then compared with ground truth segmentations available for all images in the dataset specifically collected for this purpose by the Network Dissection authors. The class with the highest intersection over union [57] of the masks is chosen as interpretation for the neuron, provided that the intersection is not too small.

The idea to see how individual neurons are connected to each other was directly inspired by [44], where it is proposed that there exist interpretable subgraphs ("circuits") that implement a part of network behaviour. While in [44] the connections

to be examined are chosen heuristically by the magnitude of the weights, with our sparse network we can not only inspect all connections for a neuron but also be sure that there is nothing else that influences its function.

In the second part of this thesis we investigate importance of small activations. Minimal activations were already inspected in [11, 45], with a caveat that there activations before ReLU were inspected with the assumption that the information will be removed by ReLU. We, on the other hand, mostly inspect small activations after ReLU and argue that there is useful information in them.

The idea that neurons correspond to individual concepts is an inviting but not yet proven assumption. Indeed, it was shown that a CNN may deliberately construct polysemantic neurons that correspond to several unrelated features [45]. Note that in polysemantic neurons multiple features appear in large activations, whereas we propose that additional functionality may be present in small activations.

In order to understand the importance and meaning of individual neurons and connections, they can be ablated by being set to zero [39]. The resulting effects on global or class-wise performance can then be examined [72].

In section 4.1 we introduce a measure of class importance for a given neuron. The main difference from the existing measures [43, 72] is taking small activations into account.

3 Learning sparse network structure

This chapter presents the algorithm for learning sparse network structure. Firstly, we formulate the problem in section 3.1. Then we present our algorithm in section 3.2, followed by experiments and discussion in section 3.3. Finally, the observations made on the trained sparse network are shown in section 3.4.

3.1 Problem statement

Let us start with a simple example: suppose we have a single convolutional layer that has N_{inp} input channels and N_{out} output channels, and the width and height of the filter are $filt_w$ and $filt_h$. Then the weight tensor $W \in \mathbb{R}^{N_{out},N_{inp},filt_w,filt_h}$. Removal of an individual connection from input channel i to an output channel j can be achieved by setting $W[j, i, :, :]$ to zero, thus effectively removing the corresponding kernel.

Learning sparse structure corresponds to learning a binary mask tensor $M \in \{0, 1\}^{N_{out},N_{inp}}$. Element-wise multiplication of it with W at every spatial position leads to removal of the connections which are set to 0 in the mask. This operation is very efficient as it can be performed in modern frameworks for tensor operation by simply extending M to 4 dimensions and performing a single element-wise product.

Note that the unit of removal in this approach is an individual connection, and not a neuron or a weight. This is helpful for learning understandable networks that nonetheless reach good performance: firstly, removing individual connections is more flexible than removing whole neurons, and secondly, it leads to clearer interpretation than removing individual weights in the kernel.

Now let us extend the problem statement to a sequential network of several layers. Nonlinearities and layers other than the convolutional (e.g. Batch normalization [23]) are not relevant for the problem statement, therefore let us represent a

network of depth N_{depth} as $\{W^i\}_{i=0}^{N_{depth}-1}$, with $W^i \in \mathbb{R}^{N^i_{out},N^i_{inp},filt^i_w,filt^i_h}$. Then the masks can be represented as $\{M^i\}_{i=0}^{N_{depth}-1}$, with $M^i \in \{0,1\}^{N^i_{out},N^i_{inp}}$.

Residual networks [18] require additional care due to their structure (a network of such architecture is separated into blocks of several layers, with every block's output being summed with the block's input passed to it by a skip (also called "residual") connection). There are two types of residual connections: identity connection and projection connection. Identity connections — the ones where the block input is simply summed up with the block output (which can be done if the dimensionalities of the input and the output tensors are the same) — do not require additional handling, because they essentially connect one input neuron to one output neuron each. Projection connections, on the other hand, include convolution of the block input to make its dimensionality the same as the block output. Therefore such projections connect all input neurons to all output neurons, as is the case for any convolution operation. To encourage the network to use the same neurons both in operations within the block and in operations of the skip connection, the mask is shared between the first convolution of the block and the projection connection (i.e. the same mask is applied in both places).

The problem of learning sparse network structure then becomes the problem of learning the binary masks $\{M^i\}_{i=0}^{N_{depth}-1}$.

3.2 Proposed algorithm

Learning discrete values via backpropagation is far from trivial (for an overview of existing approaches see section 2.1). We do not learn binary masks directly, instead we parameterize them with real-valued tensors of the same shape $\{\tilde{M}^i\}_{i=0}^{N_{depth}-1}$, $\tilde{M}^i \in \mathbb{R}^{N^i_{out},N^i_{inp}}$.

The masks are initialized with ones, which means that initially all connections are present. In the forward pass connections are discretized: values below 0.5 are set to 0, values above 0.5 are set to 1. The resulting binary mask is multiplied with the weight tensor of the corresponding convolution, which is then applied to the input of the convolution as usual.

For calculating gradients of the loss with respect to the real-valued masks $\tilde{M}^i$, the straight-through estimator [17] is used. Its key idea is passing the gradient received from the next operation directly to the previous operation without change. This

corresponds to treating the discretization step as if it were the identity function. The advantages of this estimator are conceptual simplicity and good performance [7].

This formulation of the algorithm can already be used for training a network with any standard automatic differentiation framework. But while some connections are indeed removed, the vast majority remains. Since our goal is to train a network with a small enough number of connections so that a human could inspect them manually, an additional sparsity regularizer is added as a supplementary objective during training.

If we define $L(y_{pred}, y_{true})$ to be the original training loss of the network based on the network predictions y_{pred} and ground-truth labels y_{true}, let us define the new loss as

$$L'(y_{pred}, y_{true}, \{\tilde{M}^i\}_{i=0}^{N_{depth}-1}) = L(y_{pred}, y_{true}) + \lambda \cdot \sum_{i=0}^{N_{depth}-1} L_1(\tilde{M}^i)\,, \tag{3.1}$$

where $L_1(\tilde{M}^i)$ corresponds to the L_1-norm of the real-valued mask of layer i, and λ is a hyperparameter that determines the strength of the sparsity constraint.

Note that the original loss L can include arbitrary regularization terms (e.g. regularizing network weights), as long as they do not depend on the mask parameters $\{M^i\}$.

Also note that the approach of regularizing masks before discretization has an important advantage of not influencing convolution weights' magnitudes. As a real-valued mask value for a specific connection decreases, the network cannot simply compensate for it by increasing the magnitude of the weights, since the discretized mask stays at 1. And when the real-valued mask passes below the threshold of 0.5, the connection is instantly removed.

An additional trick we use to improve the connection learning process is *connection comeback*. It is introduced to allow recovery from erroneous removal of connections. This is a possibility because in order for sparse connectivity to be learned faster, regularization is set to be strong, which may lead to fast removal of important connections. For this reason, all real-valued mask values that are disabled (i.e. are below 0.5) are increased by a small "bonus" term, which is constructed from two parts. The first one is the bonus itself, which is smaller when the mask is farther from the threshold (with the intuition being that the network strongly prefers to leave this connection disabled), the formula for it is $0.00025 - 0.0005 \cdot (0.5 - \tilde{M}^i_{disabled})$, where 0.5 is the threshold and the other values were set by manual tweaking so

that the connections would not return too quickly nor too slowly. The other part is the attenuation term for every connection, which gets initialized to be a tensor of ones with the same shape as $\tilde{M}^i$. After adding to the connections the product of the bonus and the attenuation terms, the attenuation term is multiplied by 0.999 in the places corresponding to the connections that just got the bonus. Thanks to this, the bonus becomes smaller over time, meaning that a connection does not come back infinite amount of times, while still allowing for connections to be returned several times after deletion in case the network made a mistake in removing them. This is exactly what we observed empirically.

Both normal network parameters and the masks can be optimized jointly by the same optimizer, however we found it simpler to optimize masks by a separate optimizer. This way the learning rates can be tuned independently.

3.3 Experiments in network training

In our experiments we train models for predicting 40 facial attributes in the CelebA [34] dataset. The model backbone is shared across all tasks and has ResNet-18 [18] architecture without the last classification layer. Each task has an individual decoder that consists of a single fully-connected layer of 512 neurons that maps the backbone output to the two outputs that after applying softmax correspond to probabilities of the face attribute being absent or present.

We follow the established data separation into train set (162,770 images), validation set (19,867 images), test set (19,962 images). The validation set is used for hyperparameter selection and for early stopping. Reported results were achieved on the test set. Hyperparameters are reported in the appendix section A.1.

To run experiments faster, we downsample images to the resolution of 64 by 64 pixels, and — where possible — initialize the weights with the weights pre-trained on ImageNet [12] that are available in the torchvision package [37].

Pytorch [47] is used for implementation, the code is publicly available at [1].

Previous work on CelebA [22, 52] used average accuracy of all attributes to succinctly sum up the performance of a model, however we argue that this is misleading, since most attributes are at least somewhat imbalanced. For example, "Wearing hat" attribute is true only for 4.8% of the images, "Bald" — for 2.2%, while "No Beard" is true for 83.5%. A better measure would be the average balanced accuracy [10]. In

[1] `https://github.com/AwesomeLemon/structure-for-interpretability`

order to have a point of reference to the previous work we report macro average accuracy in addition to the average balanced accuracy.

We also report the percentage of remaining connections. It should be noted that we do not learn connectivity for the first two layers, since they extract well-known low-level features [29] (e.g. Gabor filters) and because the small number of neurons (64) implies a small number of connections (256), which therefore do not contribute much to the total connection count. It also helps reduce the number of neurons being visualized.

Information about various models is provided in Table 3.1. The **baseline** model, provided for reference, has an average accuracy of 90.9%, which is not far from the state-of-the-art that is around 91%-92% [22, 52]. The next model **learned-structure-no-l1** differs from the baseline by having trainable masks. It can be seen that some small amount of connections gets removed even without the additional pressure of regularization, however this is not sufficient for our goals of manual inspection.

Model	Avg. accuracy	Avg. balanced accuracy	Connections remain	Weights remain
baseline	90.9%	79.1%	100%	11,200,192
learned-structure-no-l1	90.9%	78.7%	98.45%	11,023,055
learned-structure-l1	88.3%	73.7%	0.09%	83,156
baseline-pruned	88.5%	67.4%	—	84,738

Tab. 3.1.: Model comparison

The model **learned-structure-l1** is the key model that will be examined in the next sections. It manages not to lose a lot of performance while drastically reducing the number of connections used.

Finally, the model **baseline-pruned** is a version of the **baseline** model that was pruned to the same sparsity level as **learned-structure-l1** ($\approx$99.9% sparse) via unstructured pruning based on the magnitude of weights. The pruning was performed in three stages: 90% of the smallest weights were pruned, followed by 30-epoch-long fine-tuning, after which 90% of the remaining weights were pruned, followed by 30-epoch long fine-tuning, and finally 90% of the remaining weights were pruned (so in total 99.9% of weights are pruned, with the first two layers not being pruned for the comparison to be fair), and the model was fine-tuned for 60 more epochs. This three-staged procedure was constructed by us based on the understanding that having multiple pruning stages with fine-tuning in-between is helpful for achieving good final performance [41].

Unstructured pruning does not make the network structurally simpler, so the resulting model cannot be used for interpretability purposes. However, it is known that this method is very flexible and allows for higher sparsity [15]. Therefore we chose it as a baseline that is strong performance-wise in order to have a good reference for our uncommon setting of extremely sparse models.

The results show that even such an extremely flexible pruning method leads to a significant balanced accuracy drop in the extremely sparse setting, with our method leading to better performance as well as a structure that can be interpreted.

3.4 Examining the sparse network

In this section the model **learned-structure-l1** is inspected in various ways facilitated by its sparsity.

Firstly, all the nodes and connections in the network can be visualized in the form of a directed graph. For visualization the graphviz [16] format and xdot software [64] are used. A script is written for conversion from masks to the graph notation used in graphviz. As part of the conversion, neurons that are not connected either to input or output are removed together with all their connections. To test whether the removed connections were indeed useless, the resulting graph without them is converted back from graphviz format to masks, and evaluation is run. Accuracy is compared to the accuracy of the model that we started from, and found to be exactly the same.

At this point we can inspect how neurons are connected to each other, but it is the only thing we currently know about a neuron, together with its sequential number in the layer. Existing research [45] indicates that it can be helpful to have a visual moniker for a neuron, such as an Activation Maximization image [13]. Alternatively, we could pick dataset examples that activate a neuron the most and the least (looking at the least-activating images turned out to be quite important, results that follow from that are described in section 3.4.4).

Let us establish a notation for referring to neurons. In all the experiments throughout this thesis, we work with the ResNet-18 architecture as the backbone. This backbone has 17 layers, which we enumerate starting from zero, such that the first layer is 0, the last representation layer is 16. A neuron is referred to by its layer number and its sequential number in the layer, for examples 0_17 or 16_511. Connections are learned starting with layer 2. When we discuss several models simultaneously, we

prefix the neuron's name with the model name, for example **baseline**:16_511 or **learned-structure-l1**:0_17.

3.4.1 Circuits

Circuits can be defined as subgraphs of the neural network that seem to perform a meaningful computation together [44]. Sparse connections make searching for them easier.

For example, in the last representation layer of the **learned-structure-l1** model there seems to be an individual neuron responsible for every major hair color (see Fig. 3.1a): black (16_400), blond (16_123), brown (16_383). Grey hair does not seem to merit an individual neuron. Interestingly, the black hair neuron and the brown hair neuron are connected to all four colors of hair: inspecting the weights shows that each neuron has a positive weight to the corresponding hair color task and negative weights to all the other hair color tasks.

In the connections from the last representation layer to the task decoders, a pattern seems to emerge: a task has an input neuron specific to it (i.e. same visualization and dataset examples); and also inputs from several neurons seemingly specific to other tasks. Inspecting numerical measures that will be introduced in section 4.1 shows that those neurons most often carry at least some information about the task in question, despite it not being evident from the Activation Maximization images and dataset samples.

As an example we can look at 16_365 that is only connected to "Pale Skin" and "Rosy Cheeks". Activation Maximization images for all three can be seen in Fig. 3.1b. The similarity between 16_365 and "Pale Skin" is obvious, and this similarity is lacking in the other inputs to the "Pale Skin" task. At the same time 16_365 is also connected to the "Rosy Cheeks" task with a negative weight, therefore inhibiting the task. The fact that this connection persisted in the sparse network implies that 16_365 contains important information about "Rosy Cheeks".

Interpretable units in the last layer are perhaps not surprising, as it is intuitive that neurons there should correspond to high-level concepts. Much more exciting is an opportunity to trace the connections to the earlier layers of the network to see how the last-layer representations emerge.

In our qualitative analysis we will concentrate on a particular subgraph that shows how a feature for black hair emerges, but the analysis can potentially be performed

(a) Hair-colour circuit: shown are detectors of different hair colors in layer 16 and their connections to task decoders.

(b) 16_365 excites "Pale Skin" and inhibits "Rosy Cheeks".

Fig. 3.1.: Examples of simple circuits.

for any feature. The "Black Hair" feature seems intriguing because it shows both black hair and mouths in its Activation Maximization visualization (see 16_400 in Fig. 3.1a). This raises the question of how such a combination emerged.

The relevant subgraph is depicted in Fig. 3.2. The inspection will be made from right to left. The only input of 16_400 is 14_400 (via identity skip connection), which in turn has as its only input 13_104. These 1-to-1 connections imply that the feature is mostly ready by this point and is being simply passed deeper into the network. Activation Maximization visualizations corroborate this, as they are quite similar.

Fig. 3.2.: Multi-layer circuit for "Black Hair".

The neuron 13_104 has three inputs: visualization of 12_188 seems to depict both black hair and mouths, while 12_86 and 12_160 appear to be different textures of black hair. 12_160 has 10_160 as a single input. Finally, 12_86 is constructed from 11_124 (which is itself constructed from neurons with apparent hair texture) and 10_86, which looks closer to open mouths than to any sort of hair texture.

Based on the description above we can conclude that open mouths probably appear due to connections from 12_188 to 13_104 and from 10_86 to 12_86, with other neurons not exhibiting mouth texture. This suggests that mixing of mouths and black hair is not an inevitability. This will be further investigated in the next section where it will be attempted to disable some connections and see what happens to the Activation Maximization visualizations and output probabilities.

3.4.2 Removing connections to test circuits

While the visual exploration of the connectivity graph can be suggestive, it can also lead to misinterpretation (for example, a connection may correspond to a kernel that is close to zero and so produces almost no influence). In order to support the idea that the found circuits indeed operate as suggested, causal evidence is desirable.

One way to obtain such evidence is to see how removing individual connections impacts network behaviour. Due to the small number of connections, removing several of them leads to visible results, unlike in the dense networks.

However, there is a question of how to disable a connection. The established approach to ablating a neuron is making it always output activation equal to zero [39]. But this means that the postsynaptic neuron suddenly starts receiving (much) less input than it used to during the training, corresponding to the distribution of its activations changing. Combined with ReLu activation function that cuts off everything below zero, it may lead to a loss of valuable information.

A reasonable alternative is to replace a neuron's output with empirical mean, calculated over the training set. This still lets no information pass from the presynaptic neuron to the postsynaptic neuron, but it is unlikely to significantly shift the distribution of activations. Note that since we are interested in removing individual connections rather than neurons, we replace with zero or emprical values the activations of the presynaptic neuron that go to the specific postsynaptic neuron, while keeping activations going to the other postsynaptic neurons unmodified.

We can compare the two ablation approaches by inspecting changes in the probability of an attribute after removing some connections.

Returning to the black-hair circuit from section 3.4.1, we can disable the mouth-related connections identified previously, and observe what happens with probability of black hair on a subset of 10,000 images with black-haired people, as well as how the Activation Maximization image changes.

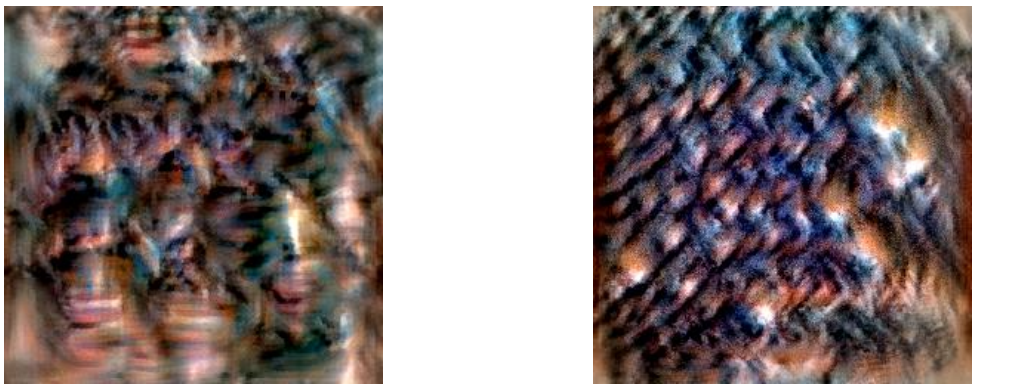
Replacing the activations with zeroes leads to black hair being hardly ever predicted, with the average probability of black hair decreasing from 0.61 to 0.24. At the same time, replacing with empirical mean leads to a reasonably small change in the probability: from 0.61 to 0.57. Therefore we conclude that ablation with empirical mean seems to be less destructive in our case of an extremely sparse network, consequently it is used for the next experiments.

To see if the black-hair circuit was identified correctly, a connection in its bottleneck can be ablated: for example, by ablating a single connection from 14_400 to 16_400, we can stop information from flowing through the circuit, thereby decreasing the balanced accuracy for the "Black Hair" task on the test set from 0.80 to 0.50 (equivalent to random guessing).

On the other hand, when we disable the connections that were identified to be mostly related to mouths rather than black hair (from 12_188 to 13_104 and from 10_86 to 12_86), the balanced accuracy for "Black Hair" changes from 0.80 to 0.77. This means that even though some useful information about black hair was passed through those connections, they were ultimately not very important for classifying black hair. For completeness we check the change in "Mouth Slightly Open": balanced accuracy for this task drops by less than 1 percentage point, which is not surprising, considering that we only disabled connections that lead to "Black Hair", while these same neurons can still pass information to the "Mouth Slightly Open" task by other paths.

But what about mouths in the Activation Maximization visualizations? In Fig. 3.3 it can be seen that when starting from noise, mouths are generated when no connections are removed, and not generated when just two mouth-related connections are disabled. This evidence is causal because every facet of the image appears because it increases the activation of the neuron (in this case — the output neuron for the "Black Hair" task).

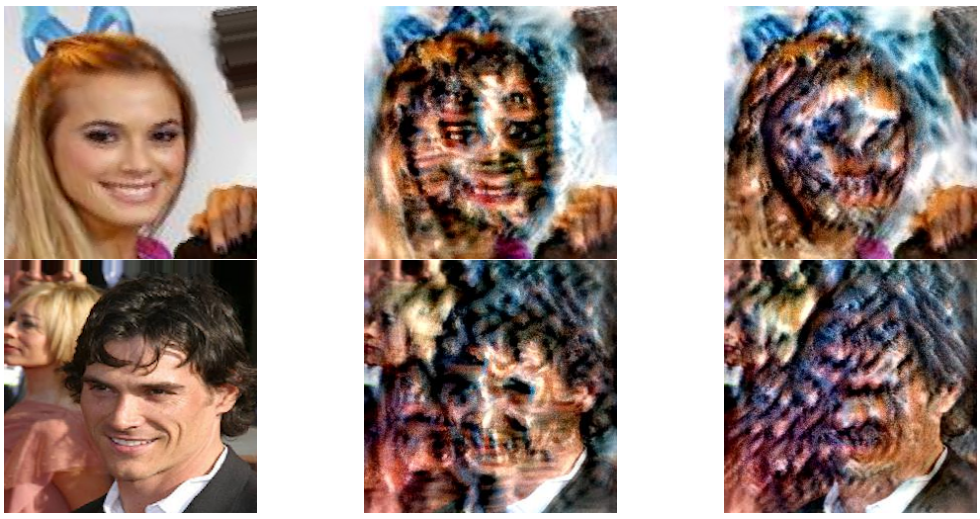
Besides the usual initialization with random noise, initialization with natural images is considered. It is done because interpreting Activation Maximization visualizations is not easy, and it may be possible that what we interpret to be mouths is actually something else. When starting the optimization with a natural image of a person, it



(a) No connections removed **(b)** Two mouth-related connections removed

Fig. 3.3.: Activation Maximization initialized with noise.

is possible to see whether the mouth is emphasized or whether some other facial feature starts to stand out. The results of such visualization are shown in Fig. 3.4.



(a) Initialization **(b)** No connections removed **(c)** Two mouth-related connections removed

Fig. 3.4.: Activation Maximization initialized with natural images.

It can be seen that when no connections are removed, not only the person's original mouth gets emphasized, but also additional mouths are drawn. The situation is opposite when the two mouth-related connections are removed: not only no additional mouths are drawn, but even the original mouth is removed.

The visualizations produced with Activation Maximization lend support to our interpretation of the connectivity patterns of neurons related to mouths and black hair.

3.4.3 GANs as interpretability tools

Testing circuits in the sparse network

To further test our understanding of connectivity, we considered utilizing GANs capable of modifying face attributes. The motivation behind it can be described like so: suppose a hypothesis to be checked is "open mouth makes prediction of black hair more likely". In that case, if images with open mouths could be modified by a GAN to only have the mouths closed with every other facial feature staying the same, then probabilities of black hair should decrease.

There is ongoing research into GANs capable of modifying individual facial attributes, with notable examples including AttGAN [19] and RelGAN [63]. We experiment with using these two models. AttGAN takes as input both an image and a vector describing presence or absence of every attribute. We find that the resulting images are often substantially changed, and the process resembles reconstruction of an image by its attributes rather than modification of a single feature. RelGAN authors noticed the same problem, and so their model takes as input an image and a vector describing the direction of change for every attribute. For example, if the goal is to open mouth and to make a person old, the relative attributes vector will have "+1" in the position for "Mouth Slightly Open", "-1" in the position for "Young", and "0" everywhere else. Visually, results produced by RelGAN look more plausible and change less about the image, so we use RelGAN for the following experiments.

Returning to the black-hair circuit, the questions are whether an open mouth increases predicted probability of black hair and whether we correctly identified the location of this influence taking place. To resolve these questions, the following experiment is conducted:

1. Select 10,000 images where the hair is black and the mouth is closed (based on the ground-truth labels)
2. Generate for each image a modified version with an open mouth (via a GAN).
3. Using the full sparse network, compare probabilities before and after the GAN transformation.
    a) Observe that probability of “Mouth Slightly Open” increases the most (i.e. the GAN worked as it was supposed to).
    b) Observe that the average probability of “Black Hair” decreases: from 0.61 to 0.51.

4. Disable the previously identified connections that are related to open mouth.

5. Using the sparse network with these connections disabled, compare probabilities before and after the GAN transformation

    a) Observe that average probability of “Black Hair” decreases: from 0.57 to 0.49.

Note that the results of this experiment cannot be used due to the reasons described later in this section.

It appears that disabling mouth-related connections changed the probability of black hair only slightly in the condition with the closed mouth (probability on the images altered by the GAN are 0.51 for the full sparse network and 0.49 for the sparse network with disabled connections).

At the same time, disabling those connections changed the probability of the black hair when the mouth was opened (probability on the original images are 0.61 for the full sparse network and 0.57 for the sparse network with disabled connections).

This is consistent with our prediction, but what happens if the experiment is carried out in the other direction: given images with black hair and open mouths, close them with a GAN and see whether the probability of black hair increases?

Running the experiment with the same steps as the one described above and the only change being that the GAN modifies images to close the mouth instead of opening it, we surprisingly observe that this also decreases the probability of black hair (from 0.62 to 0.46 in the full sparse network, and from 0.52 to 0.45 in the sparse network with disabled connections).

To understand the reason for this, we take the 10,000 images with black hair and closed mouth, use the GAN to open mouths, and then use the GAN on the already modified images to close the mouths. This operation should not change probabilities of attributes, since we introduce change and then remove it. However, we observe that the average probability for "Black Hair" decreases in both steps: from the original 0.61 to 0.51 after opening the mouth to 0.38 after closing it again, to the total drop of 0.23, which is quite far from the ≈ 0 insignificant difference that was expected.

Interestingly, if we run the images through the RelGAN with no changes required, probabilities of attributes do not change, which is exactly what should happen. This means that the GAN does not simply make hair less black every time it is

used, instead hair becomes less black when the value for the "Mouth Slightly Open" attribute is changed.

This behaviour of a GAN makes our experiments that have the goal to see how the state of the mouth influences prediction for "Black Hair" meaningless because it becomes impossible to differentiate between the following two explanations: "when we changed attribute A, probability of attribute B changed because A and B are somehow dependent in the network" and "when we changed attribute A, probability of attribute B changed because of the side-effects of using the GAN".

Therefore, the result we got that seemed to be consistent with our understanding of circuits needs to be completely discarded. The experiment is discussed here in the hope that it can potentially be reproduced in future work with some yet non-existent GAN architecture that would be able to change attributes truly independently.

Comparing sparse and dense networks

A use case for GANs separate from the one described above is to double-check results arrived at by the sparse network, with the dense network.

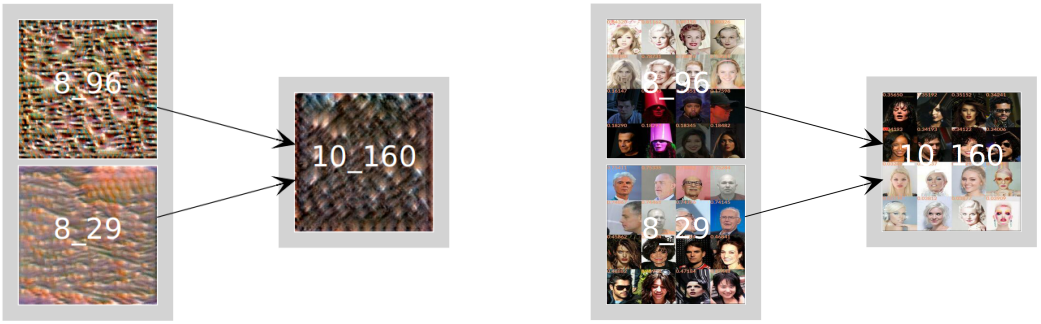
For example, using GAN images in the sparse network it was observed that the predicted probabilities for the "Big Nose" and "Bags Under Eyes" tasks followed the probability of the "Mouth Open" task: when its probability increased (i.e. GAN was used to open mouths), their probabilities increased, and when its probability decreased (i.e. GAN was used to close mouths), their probabilities decreased (the probabilities in question are average probabilities on the 10,000 images described in the previous section).

But in the dense network, probabilities of these attributes do not follow "Mouth Open" (their probabilities slightly decreased in cases of both opening and closing the mouth). Thus it can be hypothesized that these attributes follow "Mouth Open" in the sparse model simply because it is a useful shortcut when the model is forced to be sparse, and not a general interesting phenomenon.

To sum up, this was achieved by running the GAN-altered images through both sparse and dense networks and comparing the results to find that the effect present in the sparse network is absent in the dense one. Thanks to this, we can more or less safely ignore any apparent interactions between these attributes in the sparse network.

3.4.4 Small activations are meaningful

Returning to the black-hair circuit, recall that a neuron 10_160 seemed to be a pure black-hair texture detector. Its two inputs (8_29 and 8_96), however, look nothing like black hair texture (see Fig. 3.5). Inspecting positive dataset examples for them shows that these neurons are most excited by images of light-haired or bald people. However, looking at the negative dataset examples clears up the confusion, as those are dark-haired people. Thus 10_160 seems to be constructed by "flipping" its inputs, which is confirmed by the weights from both inputs being negative. This "flipping" raises a concern about the current paradigm of inspecting only maximally positive activations or dataset examples, which are not extremely helpful for understanding in this case. It also raises the question of how meaningful the information in the minimal activations of a neuron is.



(a) Activation Maximization visualizations.

(b) Dataset examples: 8 most excitatory in the top two rows, 8 most inhibitory in the bottom two rows.

Fig. 3.5.: Constructing black hair texture by "flipping" inputs.

Another interesting neuron is 16_187 (rightmost in Fig. 3.6). It attracted our attention because all the least activating images share a common feature: they depict women with bangs. This presence of meaning is unexpected: if smallest activations are images that simply lack the feature the neuron selects for, should not the dataset examples be more diverse?

But even more intriguing is the way the maximal activations of the neuron (corresponding to images of people wearing hats) are constructed through the layers. Let us trace back the connections: 16_187 has two inputs, with the one that is obviously selective for hats being 15_204 that has several inputs, none of which has images of hat-wearing people as most activating dataset examples. However, one of them, 14_421, has people with hats in its least activating samples. Tracing this feature further (to 13_279, then to 12_73, then to 10_73) we can actually observe how the

neurons select least-activating images with hats worse in the shallower layers and better in the deeper layers: essentially, we see how a feature hidden in minimal activations evolves.

It would be impossible to do this if only maximal activations and dataset examples were examined. Even if we look at minimal activations but assume them to be automatically arising opposites of maximal activations, at best we would have been able to say that the "Wearing Hat" feature arises in 15_204 by "flipping" of 14_421, but we would not have been able to trace its development in the middle layers, as we just did.

Fig. 3.6.: Feature for "Wearing Hat" evolves in small activations until 14_421, after which it is "flipped" to reside in big activations.

Therefore, investigation of small activations seems like a problem that can be quite important for understanding the inner workings of the neural networks. We conduct extensive experiments on them in the next chapter of this thesis, and find them indeed meaningful, albeit with caveats and restrictions.

It should be duly noted that sparsity of our trained network proved exactly as expected in making the number of neurons manageable, and allowing us to notice the meaningfulness of small activations, which we may not have noticed otherwise and information about which seems to not be present in the existing literature.

4 Investigating small activations

The most intriguing result produced by the investigation of the sparse network in the previous chapter is the apparent presence of meaning in small values of activations, which runs contrary to the established understanding of neurons in the networks with ReLU non-linearities, where a neuron is seen as responding to some feature if it activates highly for it.

In this chapter the phenomenon is thoroughly examined and found to affect not only our sparse network trained in the multi-task regime but also dense networks, trained both in multi-task and single-task regimes.

4.1 Measuring shifts of activations' distributions

Recall that the neuron 16_187 attracted our attention in section 3.4.4 by having all its minimally activating dataset examples being exclusively pictures of women with bangs. While dataset examples are a good hint at neuron behaviour, they could also be misleading: they could appear to share some feature just by random chance.

Therefore it would be more prudent to look at how a neuron reacts to a big number of images, and how a neuron's activations are distributed for subsets that share some attributes. Let us give two intuitive informal definitions of the distribution behaviour that could be seen.

We say that an attribute is **zero-shifted** for a given neuron if activations of images with that attribute are closer to zero than activations of images without the attribute.

Conversely, an attribute is **max-shifted** for a neuron if activations of images with that attribute are closer to maximum activation of the neuron than activations of images without the attribute.

In addition, an attribute can be neither zero-shifted nor max-shifted for a neuron if the distributions of activations of images with and without the attribute mostly overlap.

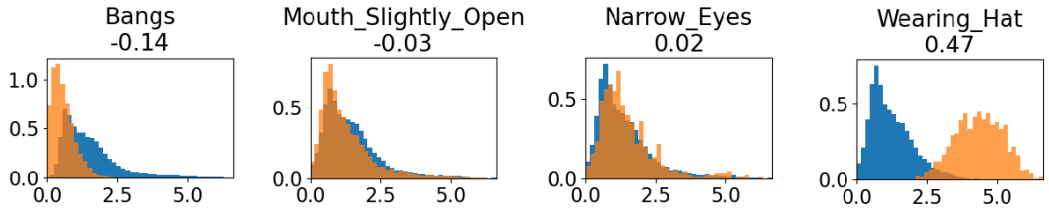


Fig. 4.1.: Normalized histograms of spatially-averaged activations of the neuron 16_187 in the sparse Resnet-18 trained on CelebA (**learned-structure-l1**). For each attribute, depicted in orange is the histogram of activations for images with this attribute, depicted in blue is the histogram of activations for images without it. Below each attribute name is the Modified Wasserstein distance between the distributions.

To get a better intuition of these informal definitions, take a look at Fig. 4.1 where distributions of activations of the aforementioned neuron 16_187 are shown for several attributes. The "Bangs" attribute is zero-shifted, the "Wearing Hat" attribute is max-shifted and the "Narrow Eyes" and "Mouth Slightly Open" attributes are neither zero- nor max-shifted in any substantial way.

The above definitions of zero-shift and max-shift are informal and reliant on visual understanding. Below we describe a quantitative approach to measuring distribution shifts.

We would like to operate with distributions of activations of a single neuron. For that, all activations of all neurons across some set of inputs (e.g. validation set) need to be stored. To unify the approach across different spatial scales, only the mean value of the feature map of the neuron is stored as an activation of a neuron on a given input.

To construct the histogram such as in Fig. 4.1 for a given neuron and a given attribute $attr$, the stored activations are separated into two sets:

1. activations of images that are predicted by the network to exhibit attribute $attr$[1];

2. activations of all the other images.

Histograms of the two sets are plotted simultaneously, with each histogram normalized to have unit area (this makes visually comparing distributions easier). Then the procedure is repeated for every neuron and every attribute.

[1]Using ground-truth labels leads to qualitatively similar results.

Finally, to measure zero- and max-shift, a modified version of the Wasserstein distance [60] is employed. Wasserstein distance is an established way to measure distances between distributions. For our case, however, it has two problems that require introduction of modifications.

Firstly, Wasserstein distance depends on the support of the distributions. For example, if two distributions span from 0 to 1, the Wasserstein distance between them will be smaller than if these distributions were to be stretched to span from 0 to 2. Since this property is irrelevant to our interest in the degree of overlap between distributions, we normalize values of all activations to lie in the $[0, 1]$ range as a preprocessing step. This leads to Wasserstein distance also being in the $[0, 1]$ range.

Secondly, Wasserstein distance is symmetric, as it does not take into account in which direction a distribution is shifted relative to the other one. We, on the other hand, would like to know whether the distribution for an attribute is zero-shifted with respect to the distribution for the other attributes, or max-shifted. Therefore a post-processing step is introduced: the calculated Wasserstein distance is multiplied by the sign of the difference of means of the two current distributions. This way, if the distribution for an attribute is zero-shifted, the Wasserstein distance becomes negative.

With both modifications the final range of the proposed measure is $[-1, 1]$ (where border values are quite unlikely, as they would correspond to one distribution being a Dirac delta at the minimum value, and the other distribution being a Dirac delta at the maximum value). In the rest of this thesis we refer to this measure either as "Modified Wasserstein distance" or "MWD".

4.2 Multi-task setting

4.2.1 Sparse network

Fig. 4.1 confirms what dataset examples hinted at for neuron 16_187: "Wearing Hat" is indeed max-shifted (MWD 0.47) and "Bangs" is zero-shifted (though the effect is smaller with MWD of -0.14). It is worth noting that in addition to the visualized attributes, several remaining attributes are max-shifted (e.g. "Bald" (MWD 0.26), "Chubby" (MWD 0.21), "Double Chin" (MWD 0.21)), and several attributes are zero-shifted (e.g. "Wearing Lipstick" (MWD -0.17), "No Beard" (MWD -0.15), "Wavy Hair" (MWD -0.15)). This means that the neuron encodes information about many attributes (not surprising in the sparse network) both in large and small activation

values, and the introduced metric allows to view the neuron functionality more fully than the dataset examples could show.

Another example discussed in section 3.4.4 was the neuron 10_160 that seemed to detect black hair texture; it had two inputs (8_96 and 8_29) that were both excited by bright images. Appropriately, for 8_96 the most max-shifted attribute is "Blond Hair" (MWD 0.14) and the most zero-shifted attribute is "Black Hair" (MWD -0.07); 8_29 seems not to be very useful as indicated by the Modified Wasserstein distances that are all close to zero, but they seem to go in the expected direction, with most max-shifted attributes being "Gray Hair" (MWD 0.04) and "Bald" (MWD 0.04), and most zero-shifted attributes being "Brown Hair" (MWD -0.03) and "Black Hair" (MWD -0.02). The neuron 10_160 indeed flips the distributions: the most max-shifted attribute for it is "Black Hair" (MWD 0.13), the most zero-shifted attribute is "Blond Hair" (MWD -0.20).

It is quite encouraging that the introduced measure seems to be predictive not only in the last representation layer, where reasoning in terms of labels makes a lot of sense but also in the middle of the network, where the features do not need to be aligned with individual attributes. This means that the methodology can be useful when trying to understand features from the middle of the network, which is an important research direction.

4.2.2 Dense networks

There is an important question of generalization to be answered: can small activations also be meaningful in the ordinary dense networks, or are they just an artifact of the sparsity constraint?

Towards that goal, the dense **baseline** from section 3.3 is examined: Modified Wasserstein distances are calculated for every neuron, for every attribute. By inspecting the results, it can be seen that in almost every neuron there is a multitude of strongly zero-shifted attributes; as an example, a neuron that is similar to the one from the section 4.2.1 can be found: in **baseline**:16_462 the most max-shifted attribute is "Wearing Hat" (Modified Wasserstein distance 0.26), most zero-shifted attribute is "Wearing Lipstick" (-0.12), which was the same in **learned-structure-l1**:16_187. Though these two neurons are not exactly equivalent (for example, "Bangs" has MWD of only -0.06 in **baseline**:16_462), they still have the same most zero- and max-shifted attributes, and similar distributions of other attributes. This implies that zero-shift can be found in dense networks too.

The example-based analysis above is in no way rigorous or complete. The proper quantitative analysis is not performed due to a problem with using multi-label datasets such as Celeb-A, namely illusory zero- and max-shifts arising due to label cooccurence probabilities. The problem is discussed in more detail in the results of the investigation of the following question: Do attributes get zero-shifted automatically as natural opposites of the attributes being max-shifted?

To clarify the question by an example: in **learned-structure-l1**:16_187 the "Bangs" attribute is zero-shifted while the "Wearing Hat" attribute is max-shifted (see Fig. 4.1); could it be that a feature that is highly activated by hats will always be least activated by bangs due to perhaps them being completely opposite to hats?

This can be easily checked by training separate networks that each predicts only one of the attributes. This way, a network trained to predict "Wearing Hat" is not influenced by the labels for "Bangs" or any other attribute, and therefore those attributes should not be either zero- or max-shifted if the shifts are learned instead of arising automatically. If they do arise automatically, however, we should see zero- or max-shifts even for the tasks the network was not trained on.

For these experiments dense ResNet-18 networks are used again, but they have four times fewer neurons in each layer to avoid overfitting. Additionally, weighted cross-entropy is used to account for class imbalance (the classes in each case being "Attribute present" and "Attribute absent"): weight for each class is $\frac{0.5}{freq_{class}}$. We also attempted using weighted cross-entropy in the experiments in section 3.3, but it did not lead to a visible effect and therefore was not used in the final reported experiments.

A separate network is trained for the two following tasks (with final performance written in parentheses): "Wearing Hat" (accuracy 0.93, balanced accuracy 0.97), "Wearing Lipstick" (accuracy 0.91, balanced accuracy 0.90). Neurons in the last representation layer are examined. Note that for calculating MWDs labels are used instead of predictions, since predictions are unavailable for the 39 attributes other than the target one.

Firstly, the "Wearing Hat"-network is inspected. Interestingly, in any individual neuron the target "Wearing Hat" attribute is about as likely to be zero-shifted (70 neurons out of 128) as max-shifted (58 neurons out of 128). This means that in about half of the cases looking at biggest activations would not be helpful (as they correspond to not wearing a hat, which is presumably hard to deduce by looking at

dataset examples without knowing that predicting hats is the only thing the network was trained on).

We would like to measure the average shift size. This should be done separately for neurons where "Wearing Hat" is shifted in different directions (otherwise average values become zero as positive and negative MWDs cancel each other).

For "Wearing Hat" mean MWD over zero-shifted neurons is -0.21 (std 0.08), mean MWD over max-shifted neurons is 0.29 (std 0.10). Thus the distribution of activations for "Wearing Hat" is significantly different from the distribution for "not Wearing Hat" both when the "Wearing Hat" distribution is zero-shifted and max-shifted.

Recall that the initial motivation for this experiment was investigating distributions for the non-target attributes. As expected, they almost do not exhibit either zero- or max-shifts. Procedure for calculation is the same as before: for every attribute, calculate mean zero-shift on the neurons where the attribute is zero-shifted, and max-shift on the neurons where the attribute is max-shifted.

Then we can examine, which attribute has the biggest average zero-shift, and which attribute has the biggest average max-shift. The former is "Bald" with mean MWD of -0.05 (std 0.028), the latter is "Attractive" with mean MWD of 0.05 (std 0.024). Mean MWDs for other attributes have smaller absolute values. For "Bangs" mean MWD over zero-shifted neurons is -0.02 (std 0.009), mean MWD over max-shifted neurons is 0.01 (std 0.007).

These shift sizes are very small, which leads to the conclusion that "Bangs" — as well as other attributes — do not appear to be shifted automatically simply due to "Wearing Hat" being shifted. Instead, the shifts of these attributes seem to be deliberately learned by the multi-task network to encode more information in a neuron.

However, the results are quite different in the network trained to predict "Wearing Lipstick". There some attributes are separated almost as well as the target one. For "Wearing Lipstick" mean MWD over zero-shifted neurons is -0.21 (std 0.07), mean MWD over max-shifted neurons is 0.22 (std 0.08). For "Rosy Cheeks" mean MWD over zero-shifted neurons is -0.14 (std 0.05), mean MWD over max-shifted neurons is 0.20 (std 0.07).

In general, there are 10 attributes, for which average zero-shifted or max-shifted MWD has an absolute value bigger than 0.15: "5 o'Clock Shadow", "Goatee", "Heavy Makeup", "Male", "Mustache", "No Beard", "Rosy Cheeks", "Sideburns", "Wearing Lipstick", "Wearing Necktie". These attributes are far from random, instead they

are very dependent on the person's gender, which comes as no surprise, given that "Wearing Lipstick" is most often defined by the gender too (these claims are supported by the conditional probabilities of "Male" given one of these attributes: they are close to 0% (for female attributes) and 100% (for male attributes), except that out of the people with the attribute "No Beard" 30% are males and 70% are females).

Has the network actually learned anything about these labels? We would like to argue that it did not and that this is a simple consequence of attribute cooccurrence probabilities.

Our argument can be summed up like so: suppose attributes $attr_1$ and $attr_2$ often cooccur, if $attr_1$ is zero- or max-shifted, $attr_2$ will be too, because it is present on most images where $attr_1$ is present and absent on most images where $attr_2$ is absent. The same applies in the reverse case where $attr_1$ and $attr_2$ almost never cooccur: then $attr_2$ would be shifted into the direction opposite to $attr_1$.

To give an oversimplified example, suppose $attr_1$ and $attr_2$ always cooccur. For a neuron that extracts information only about $attr_1$, $attr_1$ will be max-shifted, and therefore $attr_2$ will be max-shifted (since the labels are the same), despite the neuron extracting no information about $attr_2$ from the image.

Let us describe our thought process in more detail for "Wearing Lipstick" and "Rosy Cheeks". To measure cooccurence frequency, conditional probabilities of attributes can be calculated based on labels.
To approximate $P(\text{"}Wearing\ Lipstick\text{"}|\text{"}Rosy\ Cheeks\text{"})$, we can divide the number of images in the validation set where both are present by the number of images where "Rosy Cheeks" are present. $P(\text{"}Wearing\ Lipstick\text{"}|not\ \text{"}Rosy\ Cheeks\text{"})$ can be calculated similarly. The first probability turns out to be 0.97, the second one is 0.44.

Suppose we have a neuron that attains high values only when a person is wearing lipstick (and the neuron is not at all influenced by rosy cheeks). Then the MWD for "Wearing Lipstick" for this neuron will be large and positive. What about "Rosy Cheeks"?

In frequentist terms $P(\text{"}Wearing\ Lipstick\text{"}|\text{"}Rosy\ Cheeks\text{"}) = 0.97$ means that 97 out of 100 images with rosy cheeks will also have lipstick. Therefore the neuron will activate highly for the vast majority of the images with rosy cheeks.

On the other hand, $P(\text{"}Wearing\ Lipstick\text{"}|not\ \text{"}Rosy\ Cheeks\text{"}) = 0.44$ means that 56 out of 100 images without rosy cheeks will not have lipstick. Therefore the neuron will activate lowly for most images without rosy cheeks.

Combining the two facts, it can be deduced that the distribution of images without rosy cheeks will be shifted towards zero, and the distribution of images with rosy cheeks will be shifted towards the maximum. This means that MWD for "Rosy Cheeks" will be big and positive. This happens despite the neuron encoding nothing about rosy cheeks by assumption.

We find this extremely problematic: when examining a network trained on CelebA we cannot be sure if an attribute is shifted because the network learned something about it, or just because the attribute has a complicated label-based relationship to some other attributes.

Looking at conditional probability matrices is unlikely to help: they only show one-to-one dependencies between attributes, but higher-order dependencies are also possible.

Note, however, that the results previously achieved for "Wearing Hat" still hold: label cooccurences are a potential source of attribute shifts, but it was shown empirically that no attribute shifts occur when a network is trained to only predict "Wearing Hat". Lack of shifts means that label cooccurences do not cause shifts in this case. This means that the shifts in the multi-task setting must also be caused not by label cooccurences, but rather arise as a deliberate result of optimization.

The finding about the influence of label cooccurences motivates us to continue the investigation in a single-task setting, where each image has only a single label, thus allowing us to sidestep the problem.

4.3 Single-task setting

For the experiments in the single-task setting we use the CIFAR-10 [28] and ImageNet [12] datasets. Resnet-18 remains as the network architecture. Hyperparameters used to train a network on CIFAR-10 are described in the appendix section A.2, the resulting network is further referred to as **cifar-net** (top-1 accuracy 92.5%). We do not train a network on ImageNet, and instead use the pretrained weights from torchvision [37], this network is further referred to as **imagenet-net** (top-1 accuracy 69.5%).

The Modified Wasserstein distance can be calculated in the single-task setting exactly as in the multi-task one, except now it will be computed for classes instead of attributes. Manual inspection of MWDs leads to results that are similar to the ones described in section 4.2.2. In the next section we focus on investigating MWDs quantitatively.

4.3.1 Do distribution shifts correspond to network behaviour?

Inspecting MWDs suggests that a neuron can be more or less relevant for different classes, depending on the magnitude of the metric. Can this relevance be quantified? Particularly important for our overarching goal of examining small values are negative MWDs that correspond to zero-shifts: do these shifts correspond to meaningful network behaviour?

We start with the last representation layer. Due to global average pooling, output of each neuron in this layer is a scalar. This output scalar is multiplied by N scalar weights to calculate its contribution to each of the N output classes. The weight scalar connecting a neuron to a class represents impact of the neuron on the class: bigger magnitude corresponds to bigger impact; positive sign means excitation, negative sign — inhibition.

The Modified Wasserstein distance of a specific class for a specific neuron also presumably represents a relationship between the neuron and the class. The MWD can also be interpreted: bigger magnitude corresponds to more class-relevant information; positive sign means that inputs belonging to the class have activations somewhat closer to maximum, negative sign means that inputs belonging to the class have activations somewhat closer to zero.

What could different MWDs predict about the weights?

Firstly, if MWD is near zero, it implies an absence of class-relevant information, and the weight to that class should also be near zero.

Secondly, if MWD is big and positive, it means that inputs belonging to the class will induce big activations, while inputs belonging to the other classes will induce small activations. This means that the weight should also be big and positive. Maximal excitation will be achieved by the inputs belonging to the class (thanks to the multiplication of big activation by a big weight), all the other inputs will induce smaller excitation.

Finally, if MWD is big and negative, it means that inputs belonging to the class will induce near-zero activations while inputs belonging to other classes will induce bigger activations. This means that the weight should be big and negative. Minimal inhibition will be achieved by the inputs belonging to the class (their near-zero activations will remain small after multiplication by a large weight), all the other inputs will induce larger inhibition.

To sum up the reasoning above, MWDs should correlate with weights: be near zero when they are near zero, be positive when they are positive, and be negative when they are negative. This correlation can be easily computed for each neuron, since, as described above, every neuron has N weights to the outputs, and N MWDs — 1 per class.

The correlation can be averaged over a layer to calculate mean and standard deviation. For **cifar-net** mean for the last representation layer is 0.84 (std 0.09), for **imagenet-net** mean is 0.77 (std 0.04). This high correlation supports our reasoning on what MWDs entail and also shows that they are predictive of network behaviour: we can indeed see how information from MWDs is actually mirrored in the weights of the last representation layer.

However, the approach of comparing with weights is only viable for the last representation layer, outputs of which are simply multiplied by a matrix of last-layer weights and passed through a softmax. Outputs of the other layers, conversely, are passed through non-linearities, convolutions, and skip connections. All of these complicated functions need to be taken into account when understanding how a neuron is relevant to a specific class. Luckily, neural networks are specifically constructed to make derivative computation easy, and we will take advantage of that by using gradients to measure relevance (this is also the foundation of sensitivity analysis [42], which is usually applied to model inputs rather than neurons in the middle of the network).

But we are not interested in the ordinary gradient of the loss function, since we are not interested in how a neuron *weight* should be changed for decreasing the **error**. Instead, we are interested in the gradient of a **class output** with respect to a neuron's *activation*. The gradient will tell us how to change the activation to increase the output probability of the class. Positive gradient means that the activation should be increased to increase the class probability, negative gradient means that the activation should be decreased to increase the class probability.

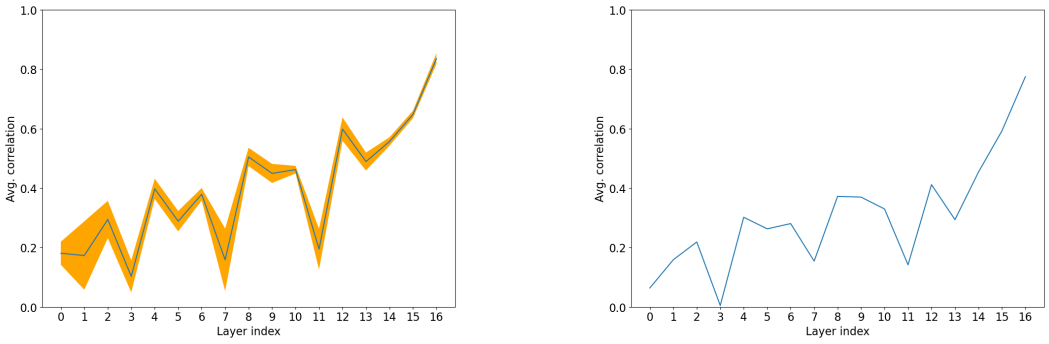
Note that for the last representation layer this gradient is exactly equal to the weight between a neuron and a class. These gradients can be considered a generalization of weight inspection to earlier layers.

Such gradients will allow us to check MWDs in the earlier layers. Indeed, if an MWD is positive, it means that the images belonging to the class induce high activations, and so the gradient should also be positive (i.e. increasing the activation increases the class probability). If an MWD is negative, it means that the images belonging to the class induce small activations, and so the gradient should also be negative (i.e decreasing the activation increases the class probability). And if an MWD is near-zero, it means that the class does not tend in any direction, and so the gradient should be near-zero too.

To measure how much MWDs correspond to gradients we again can simply calculate the correlation between them for every neuron, because for every neuron there are N gradients (one from each output class) and N MWDs (again one per class).

To summarize information further, we can calculate the average correlation per layer. For CIFAR-10 in addition to **cifar-net** we train 4 more equivalent networks with different random initializations, calculate average correlations for all of them, and then compute standard deviation across the runs. For ImageNet due to computational constraints no additional networks are trained.

The resulting correlations are visualized in Fig. 4.2.



(a) CIFAR-10 (5 networks, shaded area is ± 1 standard deviation)

(b) ImageNet (single network)

Fig. 4.2.: Average correlations of MWDs and gradients of class outputs with respect to neurons' activations.

The trend seems to be that the shallower the layer, the smaller the average correlation. This can be explained by the usage of class labels which are not as meaningful in the earlier layers as in the later ones.

All the average correlations are positive, and most are no lower than about 0.3, with some glaring exceptions in layers 3, 7, 11: there correlations experience sudden drops for every initialization. These happen to be the layers in the middle of the ResNet blocks where the skip connections are projections (i.e. include convolutions that increase input dimensionality). To check whether skip connections are indeed responsible for this inconsistency in the results, a network with ResNet-18 trunk architecture but without any skip connections is trained on CIFAR-10. Gradients, MWDs, and their correlation are computed the same way as before. It turned out that the average correlations are in fact mostly monotonic, with nothing special happening in layers 3, 7, 11, and with actual minimal average correlation value being 0.27 in the very first layer. We conclude that near-zero correlations in specific layers of ResNets arise because of interactions of gradients in the skip connections.

Additionally, we performed several sanity checks on these results. The main concern was that the correlation between gradients and MWDs may arise due to gradients across layers possibly correlating with each other, which could lead to them correlating even with random noise. To test if this is indeed what happens, we compute correlations of gradients not with MWDs, but with different variants of random data: random sample per layer, random sample that is the same across layers, shuffled MWDs. In all of these cases no trend is visible, with correlations fluctuating around zero and the maximal absolute value of the correlation being 0.06. This implies that the trend in the correlations we see is unlikely to be spurious.

To conclude, positive correlations of gradients and MWDs across layers mean that the proposed metric actually captures some aspects of the network behaviour, and the information reflected in it also influences class probabilities predicted by the network.

4.3.2 Impact of neuron removal

While high correlations hint that an effect exists, they do not provide causal evidence. For that we need an intervention: for example, removing a neuron by setting its outputs to zero. It should be noted that in section 3.4.1 we argued that a neuron should be ablated by setting its output to the empirical mean. However, such an ablation approach would lead to a less visible impact of zero-shifts on performance, therefore we ablate a neuron with zeroes, which is also done in the experimental protocol that we follow [72] (see below).

Existing work has shown [72] that ablating a single unit does not impact overall accuracy much and may even improve it. However, if a neuron is selective for a

specific class (selectivity can be measured by various metrics [43, 72] that all depend on maximal activations), removing it decreases performance for that class [72].

In the experiment that follows we use the protocol from [72], but with a different class performance measure. In [72] and [6] the authors refer to the performance measure as "single-class accuracy", which means balanced binary accuracy when classifying the target class versus every other class. If we define TP to mean True Positives, TN to mean True Negatives, FP to mean False Positives, FN to mean False Negatives, balanced accuracy is $0.5 \cdot (\frac{TP}{TP+FN} + \frac{TN}{TN+FP})$.

However, when reasoning about changes in the performance of a single class, it is more common to use Precision ($\frac{TP}{TP+FP}$) and Recall ($\frac{TP}{TP+FN}$) which provide more intuitive understanding of what is happening with the predictions. Additionally, inspecting both measures allows us to see effects that go in different directions and would cancel out in a measure such as balanced accuracy.

For the first experiment, we observe how ablating a neuron impacts class precision and recall. Specifically, we compute the correlation between maximal Modified Wasserstein distance for a neuron and a maximum drop in class precision (or recall). A positive correlation would mean that removing a neuron where a class is max-shifted actually improves precision (recall), because then bigger MWD would correspond to bigger precision (recall) change, i.e. to the change that is more positive (and positive change of the metric means improvement of the metric).

For the sake of consistency, when investigating relevance of zero-shifts, we take an absolute value of the minimal Modified Wasserstein distance for a neuron. These are correlated with the maximum drop in class precision (or recall). Positive correlation again would mean that removing a neuron where a class is zero-shifted actually improves precision (recall), because then bigger absolute value of MWD would correspond to a bigger precision (recall) change, i.e. to the change that is more positive.

Correlations of maximal MWD and absolute value of minimal MWD with the largest precision and recall drops are reported in Table 4.1.

	Maximal MWD	Minimal MWD
Largest recall drop	-0.10 (p-value 0.03)	0.13 (p-value 0.004)
Largest precision drop	-0.04 (p-value 0.42)	-0.17 (p-value 0.00007)

Tab. 4.1.: Correlations of maximal and minimal MWDs for a neuron with maximal recall and precision drops when this neuron is ablated (last representation layer).

Let us first inspect the results for Maximal MWD. The correlation of -0.10 with the largest recall drop suggests that the bigger the MWD, the more recall will suffer. The correlation value of -0.04 with the largest precision drop is most likely meaningless and occurs due to random chance, as indicated by the p-value of 0.42.

For the Minimal MWD the results are quite different: positive correlation of 0.13 with the largest recall drop means that ablating a neuron with large minimal MWD leads to a decrease in recall drop. And the negative correlation of -0.17 with the largest precision drop suggests that ablating a neuron with a large absolute value of minimal MWD hurts the precision.

These results are given more context with another experiment. The approach is to greedily disable neurons relevant to a specific class and see how it impacts precision and recall of that class. For a given class we sort neurons in three ways:

1. sort neurons in descending order by MWD for the target class (**sort-max**);
2. sort neurons in ascending order by MWD for the target class (**sort-min**);
3. sort randomly (**random**).

Then the first 1, 4, 8, 16, 32, 64, 128 neurons in the last representation layer are removed according to the appropriate order, and impact on the class precision and recall are recorded. Changes are then averaged across classes for each amount of removed neurons and plotted in Fig. 4.3 together with the standard deviations of the changes that are also computed across classes.

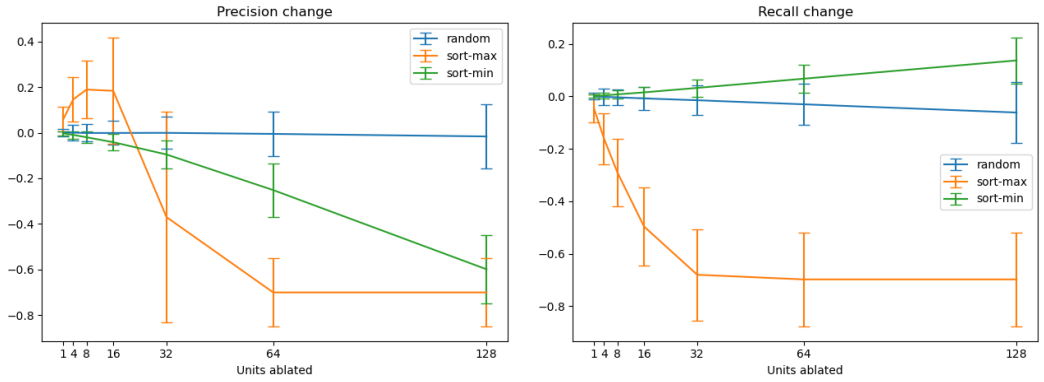


Fig. 4.3.: Average changes in precision and recall during greedy ablation of neurons for a class based on variously sorted MWDs.

It can be seen that precision and recall behave quite differently. This behaviour, however, can be easily interpreted given our understanding of MWDs and last-layer weights (see section 4.3.1).

Let us start with precision. Removing random neurons seems not to produce much impact. Removing neurons starting with most zero-shifted consistently decreases precision. Firstly, this shows again that zero-shifts are important for network performance. Secondly, this result can be easily explained by our results from section 4.3.1: there it was found that MWDs strongly correlate with last-layer weights, which entails that when neurons where the class is zero-shifted are removed, we remove inhibition, as those neurons are likely to be connected to the class output by negative weights. Because of the removal of inhibition, the amount of False Positives increases drastically, which leads to a drop in precision.

Similarly, when the neurons where the class is max-shifted are removed, excitation is removed, as those neurons are likely to be connected to the class output by positive weights. At first this leads to an increase in precision because only the examples the model is most sure of get classified as the target class. When a larger amount of excitatory neurons is removed, however, nothing is classified to belong to the class, which makes precision quickly drop to zero (actually, since both TP and FP are zero, precision is undefined, but we interpret it as zero).

The big standard deviation of precision change when 32 max-shifted neurons are removed is indicative of a transition from the first regime to the second one. If information about a specific class is present in many neurons, the removal of 32 neurons leads to an increase in precision. But if information about a class is present in few neurons, after removing the 32 neurons most relevant to the class, the class is not predicted anymore, and the precision drops drastically. Given that the average precision change is negative, the majority of classes are encoded by less than 32 neurons (five classes that lose the most precision are "flamingo", "yellow lady's slipper", "lycaenid", "sorrel", "monarch"). But there are still classes that rely on more than 32 neurons and that correspondingly experience increases in average precision (five classes that gain the most precision are "restaurant", "miniature poodle", "tiger cat", "maillot", "ladle"). This opposite behaviour in different classes explains the large standard deviation. It should be noted that the classes that lose the most precision do not seem to be any more similar to each other than to the classes that gain the most precision, and vice versa.

The impact on the recall can be analyzed similarly. Removing random neurons leads to a very slight recall decrease. Removing neurons starting with the most zero-shifted ones actually increases recall, since, as determined above, their removal corresponds to the removal of inhibition: as more and more inhibition is removed, more and more samples are classified as belonging to the class (number of true positives increases, number of false negatives decreases). On the other hand, removing

neurons starting with the most max-shifted ones removes excitation, which starkly reduces the number of true positives, and therefore leads to a decrease in recall.

4.3.3 Interpreting individual neurons

In the previous sections, the impact of small activations was established in numerical terms, by observing how zero- and max-shifts are related to gradients and how ablating zero-shifted neurons impacts class-wise precision and recall. In this section we investigate whether small activations are helpful for understanding individual neurons in dense single-task networks.

The presence of meaning in small activations was originally noticed by us when looking at minimally activating dataset examples in the multi-task setting (see section 3.4.4). Maximally and minimally activating dataset examples are used in the singe-task setting too. It should be noted that we select minimally activating dataset examples based on their activations after ReLU, not before.

Another useful visualization of the behaviour of an individual neuron is Activation Maximization (described in section 2.2 and used in section 3.3). It allows for generating an input that maximizes the activation of a neuron via an optimization process. Since our interest lies with small activations, the optimization process can be modified to **minimize** the activation of a neuron. This idea was introduced in [45] as a way to give context to maximal activations (the method was not given a name, for convenience we will refer to it as "Activation Minimization"). Since the optimization was performed on values before ReLU, its result represents the negative part of the spectrum that will be completely removed by ReLU.

We, on the other hand, are arguing in this thesis that minimal activations are important behaviourally, i.e. even after negative values are removed by ReLU. Therefore, we attempt to use the same procedure after ReLU, meaning that the minimal value achievable by optimization is zero, whereas previously there existed no bound. This makes optimization quite complicated and the results not very good, but even non-pure results can be interpreted. For making interpretation easier the results are compared with pre-ReLU Activation Minimization.

In addition to dataset examples and Activation Maximization we utilize Network Dissection [5] to examine small activations. Network dissection is an established approach that allows finding neurons that activate strongly for a specific feature. Since we are interested in weak activations, an intuitive idea is to "flip" the algorithm. The original algorithm selects a threshold for segmentation such that only 0.05%

of neuron activations are above this threshold, and considers values above the threshold as segmentation mask. It is trivial to "flip" the algorithm by selecting a threshold such that only 0.05% of neuron activations are below it, and to consider values below the threshold as the segmentation mask.

Both the original and the flipped approaches require a long tail of extreme values. A long tail is usually present in after-ReLU positive values, as well as in pre-ReLU negative values, for which the flipped method turns out to be applicable without any hyperparameter change. However, ReLU completely cuts off this tail of negative values, making the application of the flipped algorithm problematic. Nonetheless, we find that making the threshold weaker (from 0.05% to 5%) allows the algorithm to interpret a small number of neurons. This happens because these neurons are practically unaffected by ReLU, with only a small percentage of their activations being made equal to zero. This corresponds to the distribution of the non-spatially averaged activations being Gaussian instead of exponential.

We use the implementation of the Network dissection algorithm provided by the authors[2]. It works very well on the **imagenet-net** (the ResNet-18 pretrained on ImageNet from section 4.3.1), but unfortunately the algorithm fails on the **cifar-net**, which we attribute to the small spatial size of CIFAR images: most likely, the spatial resolution is simply not good enough for accurate segmentation maps to be constructed from the feature maps.

Consequently, **cifar-net** is investigated only via dataset examples, Activation Minimization, and Modified Wasserstein distances.

Let us start with examples showing that after-ReLU small values can be meaningful. For that we select neurons with large negative MWD and show for them maximally and minimally activating dataset examples, Activation Maximization visualization, pre-ReLU and after-ReLU Activation Minimization visualizations. In Fig. 4.4 three neurons from different depths of the network are shown: 2_4, 7_81, 15_243 (for transparency it should be noted that the last representation layer is not shown because the distributions there are always exponential, which leads to more than one class being zero-shifted, which makes visualizations harder to understand).

The neuron 2_4 seems to be most activated by bright blue or red colors, preferably vertically oriented (which can be seen in the Activation Maximization image, but not so much in dataset examples). It is least activated by images of frogs, but not due to a frog shape, rather by the texture of diagonal lines which is often present on the frogs. This is clearly visible in the pre-ReLU Activation Minimization image,

[2] `https://github.com/CSAILVision/NetDissect-Lite`

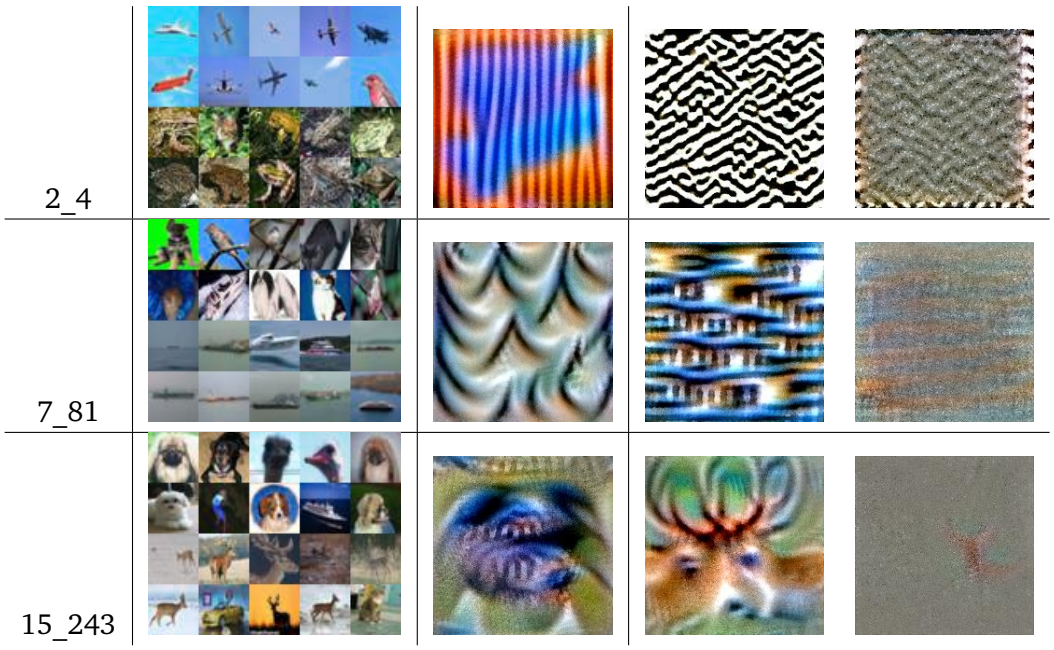


Fig. 4.4.: Visualizations of selected neurons in **cifar-net**. From left to right: dataset examples (10 most excitatory in the top two rows, 10 least excitatory in the bottom two rows), Activation Maximization, Pre-ReLU Activation Minimization, After-ReLU Activation Minimization.

and less clearly visible in the after-ReLU visualization. Nonetheless, after-ReLU visualization still shows the same pattern. Largest positive MWD for the neuron is 0.15 for "Airplane", the largest negative MWD for the neuron is -0.14 for "Frog".

It is not quite clear what the neuron 7_81 is most activated by. The dataset examples that lead to biggest activations include a variety of classes, with many of those examples being not very similar to the Activation Maximization image. However, if we inspect minimally activating examples, we find that they are all images of ships, which is also reflected by the blue color and horizontal lines in the pre-ReLU Activation Minimization (with the pattern being similar in the after-ReLU Activation Minimization image). This suggests an interpretation for maximal values: perhaps the neuron is excited by diagonal lines, in contrast to the horizontal lines that seem to lead to the smallest activations.

Lack of clear interpretation of maximum values is reflected in the largest positive MWD being quite a small value of 0.06 for "Cat". The largest negative MWD is -0.12 for "Ship", as expected.

The neuron 15_243 again lacks a crystal clear interpretation of its maximal activations, perhaps it could be described as selecting for animal heads. The Activation Maximization image is not at all helpful here. Pre-ReLU Activation Minimization

image, on the contrary, is very understandable in its representation of "Deer" (which can also be seen in the after-ReLU Activation Minimization image). The interpretation is confirmed by the least-activating dataset examples, with 8 out of 10 being images of deers. The largest negative MWD is appropriately -0.19 for "Deer"; the largest positive MWD is 0.13 for "Dog".

Examples above make a case that looking at small activations is helpful in understanding individual neurons of **cifar-net**. ould the same be said about **imagenet-net**?

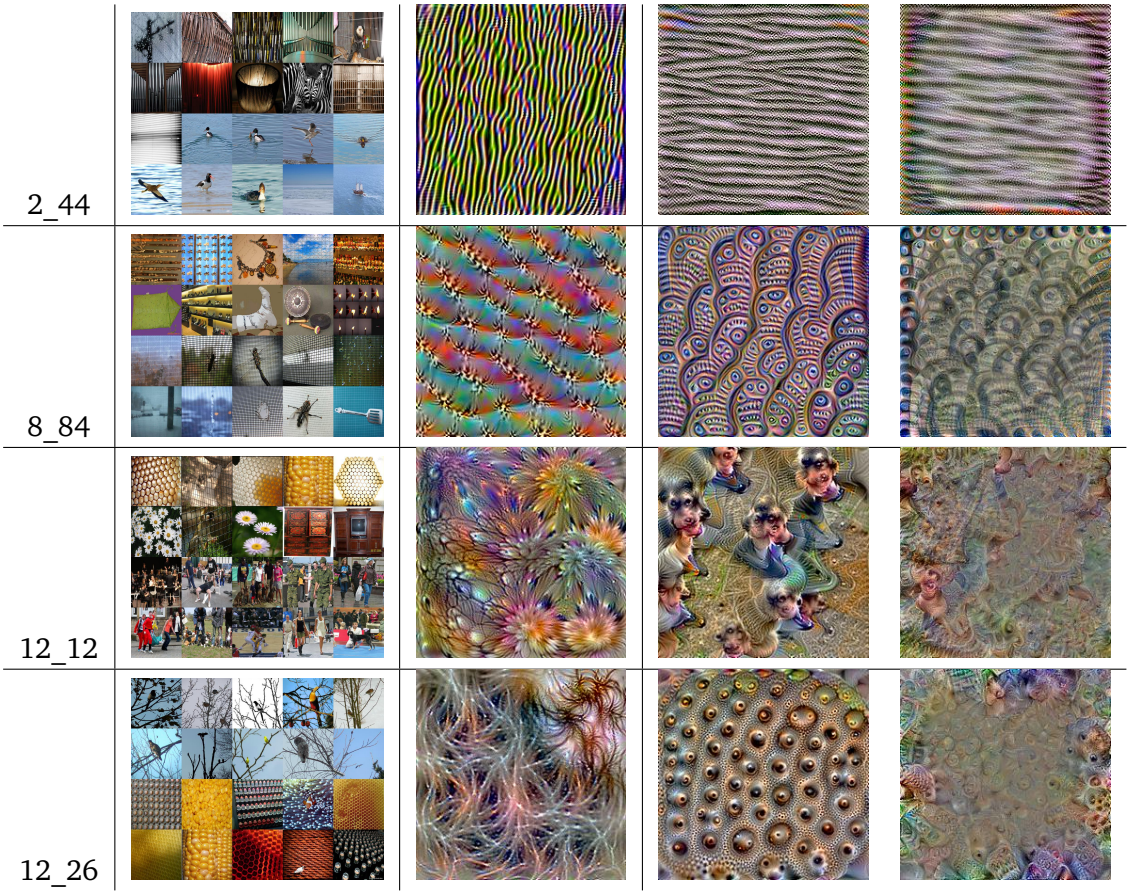


Fig. 4.5.: Visualizations of selected neurons in the **imagenet-net**. From left to right: most and least activating dataset examples, Activation Maximization, Pre-ReLU Activation Minimization, After-ReLU Activation Minimization.

Depicted in Fig. 4.5 are examples of neurons with meaningful small and big activations (2_44, 12_26), and neurons where small activations are meaningfull while big activations do not appear to be (8_84, 12_12).

The neuron 2_44 seems to respond to vertical stripes in its maximal activations and to horizontal stripes in its minimal activations. It makes a lot of sense that both ends

of the activation spectrum are used since these textures are natural opposites that are both present in natural images.

The largest positive MWD is 0.19 for "theater curtain" (which has vertical folds), the largest negative MWD is -0.15 for "red-breasted merganser". While the bird itself does not have particularly many horizontal stripes, by looking at the least activating image we can see that the bird is often swimming on the waves that have the exact horizontal stripe structure the Activation Minimization images are showing. This is a good example of the caution needed when examining early layers with MWDs that were calculated based on classes: while still somewhat useful, they do not tell us the exact feature the neuron is shifted for, but rather a class that is correlated with this feature.

The next neuron where both ends of the spectrum make sense is 12_26: maximal activations appear to correspond to branch-like structures, minimal activations seem to be responsible for the repeated structure of roundish things. Interestingly, MWDs are not very helpful in this case, with the largest positive one being "pirate" (0.16) and largest negative one being "corn" (-0.05).

For the neuron 8_84 maximal activations are hard to properly describe, while minimal activations seem to belong to images with high-frequency grid structure. Largest positive MWD is 0.12 for "cheeseburger", largest negative MWD is -0.28 for "window screen".

The neuron 12_12 again has maximal activations that defy simple explanation, with some floral pattern mixing with repeated structures and cupboards (largest positive MWD is 0.17 for "cardoon"). At the same time, smallest activations seem to correspond to images of people, and largest negative MWDs belonging to classes that are usually found near people: "basketball", "ballplayer", "volleyball", "barbell", "swimming trunks".

The minimal activations of these neurons can further be interpreted via flipped Network Dissection on pre-ReLU activations. The explanation classes found by the Network Dissection are consistent with our understanding: 2_44 is "lined" (IoU 0.04), 8_84 is "perforated" (IoU 0.05), 12_12 is "person" (IoU 0.03), 12_26 is "polka-dotted" (IoU 0.11).

However, the fact that the flipped Network Dissection was applied on pre-ReLU activations is an important caveat: there are no guarantees that those results will transfer to after-ReLU activations. In fact, the vast majority of the detectors found by flipped Network Dissection pre-ReLU (and it finds hundreds) seem to have nothing to do with neuron behaviour after ReLU.

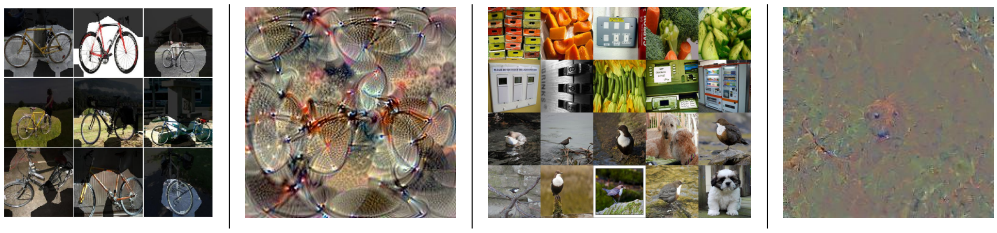


Fig. 4.6.: Visualizations for the neuron 15_101: pre-ReLU segmentation maps, pre-ReLU Activation Minimization, after-ReLU most and least activating dataset examples, after-ReLU Activation Minimization.

Let us present a very clear example. The neuron 15_101 (see Fig 4.6) is found to be bicycle detector by the flipped Network Dissection (IoU 0.18). The largest negative pre-ReLU MWD for it is -0.22 for the class "bicycle-built-for-two" (note that in all the other cases when talking about the largest negative MWD we calculate it after ReLU, not before). The pre-ReLU Activation Minimization produces bicycles.

But after ReLU flipped Network Dissection cannot work for this neuron because there are too many zeroes in its activations. Largest negative MWD is -0.04 for the class "ground beetle". After-ReLU Activation Minimization does not produce anything resembling bicycles. Minimally activating dataset examples are birds.

To conclude, all the pre-ReLU evidence suggests that 15_101 is a bicycle detector, and all the after-ReLU evidence suggests that it is not. After-ReLU evidence must take precedence, as it works with values that are directly used by the network for prediction. Therefore it must be concluded that 15_101 does not play the role of a bicycle detector.

We do not know the reason for the discrepancy. Perhaps apparent meaningfulness of pre-ReLU activations is a fluke produced by random chance. Perhaps the network would like to use that information, but cannot because of ReLU, and so the information becomes almost undetectable after it. One thing is certain: any results produced on pre-ReLU activations must be double-checked on the after-ReLU ones.

As discussed before, flipped Network Dissection faces problems after ReLU due to a lack of a long tail of activations: there are too many zeroes there. However, with the relaxed threshold, it can still find meaningful neurons after ReLU, with the prerequisite that the amount of zeroes in its activations is not too high (i.e. smaller than the 5% threshold we use in the flipped after-ReLU Network Dissection).

Unfortunately, there is another problem caused by a lack of the distribution tail: we cannot properly visualize segmentation maps for these neurons (as in the leftmost image in Fig. 4.6). In the original algorithm, the images are sorted by the magnitude

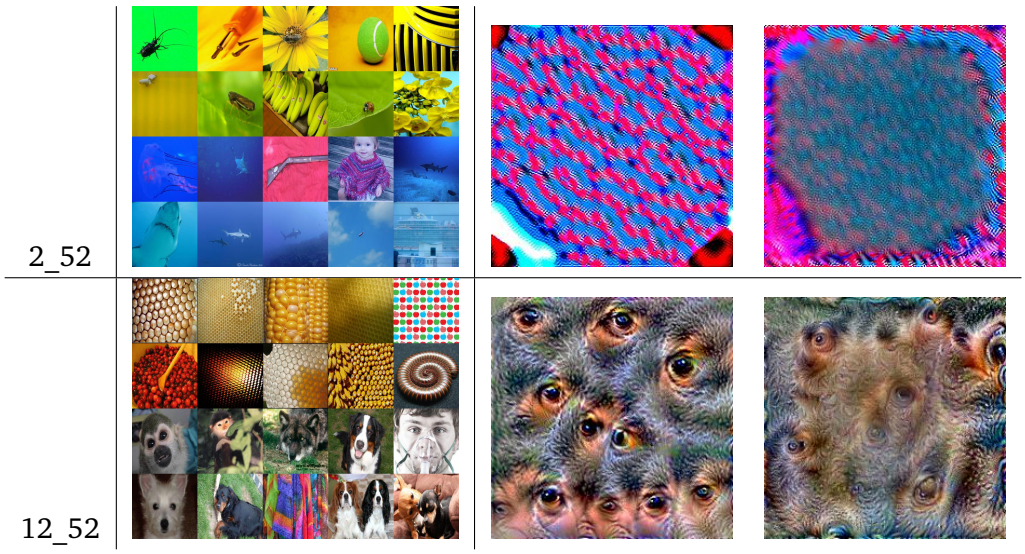


Fig. 4.7.: Visualizations of neurons in **imagenet-net** for which after-ReLU flipped Network Dissection worked. From left to right: most and least activating dataset examples, Pre-ReLU Activation Minimization, After-ReLU Activation Minimization.

of the biggest activation value in the feature map. This can be done similarly in pre-ReLU flipped Network Dissection, by sorting by the smallest activation value in the feature map. However, after-ReLU smallest activation value is zero, and it appears in a large proportion of the feature maps, often only once or twice. The resulting segmentation maps, where only a small number of pixels is part of the segmentation map, are uninterpretable.

For this reason, only dataset examples and Activation Minimization images are presented in Fig. 4.7.

The neuron 2_52 is predicted by after-ReLU flipped Network Dissection to detect "blue" with IoU 0.43. Dataset examples, as well as Activation Minimization images indeed show blue and also purple.

The neuron 12_52 corresponds to "head" with IoU 0.08, according to the after-ReLU flipped Network Dissection. This seems to be cofirmed by the dataset examples of human and animal heads. The Activation Minimization images seem to depict eyes surrounded by skin surrounded by fur or hair texture.

Thus flipped Network Dissection seem to be working reasonably after-ReLU despite the weakened threshold.

To conclude this section it can be said that various visualization techniques do not only confirm the existence of meaningful small activations, but can also be used to explore them.

Conclusion & Future work

5

5.1 Conclusion

The overarching motivation behind this thesis was to further the investigation into the inner workings of neural networks. This goal was realized two-fold.

Firstly, an approach to learning sparse interpretable network structure was introduced. It allows for manual inspection of very sparse networks that retain performance and behaviour of their big and dense counterparts, which are actually used in solving challenging problems. We consider the proposed approach of inspecting very sparse networks to be a tool that could prove very useful in the field of neural network interpretability. To substantiate this claim, we ourselves use the approach to make several observations about the inner workings of neural networks.

Our second contribution comes from one such observation, namely that small activations are meaningful in modern networks with ReLU non-linearities. This augments the common view of individual neurons and affects the way they should be examined. The generality of the phenomenon was tested in several domains with a variety of techniques: from inspecting the weights to using adapted versions of existing feature visualization approaches. The positive results of the inspections not only confirm meaningfulness of small activations but also serve to show that the sparse networks trained by the proposed algorithm could be used as models of the dense networks in interpretability experiments.

However, the proposed methodology comes with its weaknesses. For example, the algorithm for learning sparse networks requires hyperparameter tuning. Additionally, effects produced by small activations are not very large, and their prevalence is unknown. The neurons like those shown in section 4.3.3, where all the methods we use for investigation agree, are quite rare. Large negative MWD is not a sufficient condition for finding these neurons. We do not know the cause for such neurons arising in some circumstances, but not the others. This multitude of unanswered questions means that future research is open to investigations both into the methodology of using sparsely-connected networks as models and into the concrete phenomenon of meaningful small activations.

5.2 Future Work

Research can be continued both into directions outlined above and into others that we will sketch here.

In addition to making the proposed algorithm resilient to hyperparameters, it would be interesting to apply it to multi-task pruning. Because the approach allows masks to be learned together with normal parameters via gradient descent, pruning strength for different tasks can be modulated by setting bigger or smaller weight for each task. This way, more trunk parameters relevant to the important tasks could potentially survive the pruning, in comparison to the unimportant tasks, for which fewer relevant parameters will remain. Static pruning approaches relying on statistics of the weights cannot achieve this.

Minimal activations need to be scrutinized further. The magnitude of their effect on the network needs to be properly investigated. Another interesting question is why they appear in the first place. If they are an inherent feature of artificial neurons, why cannot we find them in every one? Are our methods not precise enough, or is the phenomenon simply rare? And if meaningful small activations are rare, what modulates their frequency? Do better-performing networks have more neurons with meaningful small activations, or fewer? Or perhaps smaller networks with fewer neurons are forced to use small activations of these neurons to properly fit the domain, while larger networks have enough neurons to rely only on maximal activations?

Several intriguing questions are related to the role of ReLU. Firstly, is there a meaningful difference between activations that are exactly equal to zero and the ones that are just very close to it? If there is a difference, it could be that values equal to zero correspond to patterns that are not relevant, and all the other values populate the scale spanning from the feature of small values to the different feature of maximal values.

It also follows from our work that if much useful information is lost due to the hard cut-off of ReLU (recall the pre-ReLU bicycle detector from section 4.3.3), it would be helpful not to have 0 as a hard threshold. Indeed, many modifications of ReLU (such as Leaky ReLU [36] or Mish [40]) go this way, although for entirely different reasons. Our very preliminary results on ImageNet-trained ResNet-50 with Mish activation seem to suggest that its usage indeed helps to preserve information that would otherwise be lost; specifically, flipped Network Dissection finds many interpretable neurons after the activation function without hyperparameter change, which it could

not do with ReLU. It would be interesting to see how different modifications of ReLU influence small activations.

While the Modified Wasserstein distance metric was initially proposed in this thesis for the goal of measuring the importance of small activations, it was applied to large activations just as well. Could it help with identifying neurons that are polysemantic in their large activations? Polysemantic neurons encode several unrelated concepts and were previously found by manual examination of Activation Maximization images [45]. Inspecting visualizations of every neuron is untenable. Instead, we could select a subset of neurons, where each has highly positive MWDs for several classes, and only inspect those. Unfortunately, this is unlikely to be a perfect filter because a neuron may respond to a feature that is shared across several classes: for example, a dog detector would be max-shifted for many species of dogs. Nonetheless, if developed further, this idea may make looking for polysemantic neurons manageable.

Small activations could also be inspected on tasks other than classification, for example detection or segmentation. Whether the results hold for them is an open question.

Finally, it would be interesting to know how small activations influence classes of neural networks different from CNNs, that nonetheless also utilize ReLU nonlinearity, such as fully-connected networks, recurrent networks, and Transformers [61].

Bibliography

[1] Karim Ahmed and Lorenzo Torresani. "Maskconnect: Connectivity learning by gradient descent". In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2018, pp. 349–365 (cit. on p. 3).

[2] Sajid Anwar, Kyuyeon Hwang, and Wonyong Sung. "Structured pruning of deep convolutional neural networks". In: *ACM Journal on Emerging Technologies in Computing Systems (JETC)* 13.3 (2017), pp. 1–18 (cit. on p. 5).

[3] Dmitrii Babaev, Maxim Savchenko, Alexander Tuzhilin, and Dmitrii Umerenkov. "ET-RNN: Applying Deep Learning to Credit Loan Applications". In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019, pp. 2183–2190 (cit. on p. 1).

[4] Sebastian Bach, Alexander Binder, Grégoire Montavon, et al. "On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation". In: *PloS one* 10.7 (2015), e0130140 (cit. on pp. 1, 6).

[5] David Bau, Bolei Zhou, Aditya Khosla, Aude Oliva, and Antonio Torralba. "Network dissection: Quantifying interpretability of deep visual representations". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 6541–6549 (cit. on pp. 1, 6, 39).

[6] David Bau, Jun-Yan Zhu, Hendrik Strobelt, et al. "Understanding the role of individual units in a deep neural network". In: *Proceedings of the National Academy of Sciences* (2020) (cit. on p. 36).

[7] Yoshua Bengio. "Estimating or propagating gradients through stochastic neurons for conditional computation. arXiv preprint arXiv: 1308.3432". In: (2013) (cit. on p. 10).

[8] Alexander Binder, Grégoire Montavon, Sebastian Lapuschkin, Klaus-Robert Müller, and Wojciech Samek. "Layer-wise relevance propagation for neural networks with local renormalization layers". In: *International Conference on Artificial Neural Networks*. Springer. 2016, pp. 63–71 (cit. on p. 6).

[9] Felix JS Bragman, Ryutaro Tanno, Sebastien Ourselin, Daniel C Alexander, and Jorge Cardoso. "Stochastic filter groups for multi-task cnns: Learning specialist and generalist convolution kernels". In: *Proceedings of the IEEE International Conference on Computer Vision*. 2019, pp. 1385–1394 (cit. on p. 3).

[10] Kay Henning Brodersen, Cheng Soon Ong, Klaas Enno Stephan, and Joachim M Buhmann. "The balanced accuracy and its posterior distribution". In: *2010 20th International Conference on Pattern Recognition*. IEEE. 2010, pp. 3121–3124 (cit. on p. 11).

[11] Nick Cammarata, Gabriel Goh, Shan Carter, et al. “Curve Detectors”. In: *Distill* (2020). https://distill.pub/2020/circuits/curve-detectors (cit. on p. 7).

[12] J. Deng, W. Dong, R. Socher, et al. “ImageNet: A Large-Scale Hierarchical Image Database”. In: *CVPR09*. 2009 (cit. on pp. 2, 11, 31).

[13] Dumitru Erhan, Yoshua Bengio, Aaron Courville, and Pascal Vincent. “Visualizing higher-layer features of a deep network”. In: *University of Montreal* 1341.3 (2009), p. 1 (cit. on pp. 6, 13).

[14] European Union. “Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)”. In: *Official Journal L110* 59 (2016-05-04), pp. 1–88 (cit. on p. 1).

[15] Trevor Gale, Erich Elsen, and Sara Hooker. “The state of sparsity in deep neural networks”. In: *arXiv preprint arXiv:1902.09574* (2019) (cit. on pp. 5, 13).

[16] Emden R Gansner and Stephen C North. “An open graph visualization system and its applications to software engineering”. In: *Software: practice and experience* 30.11 (2000), pp. 1203–1233 (cit. on p. 13).

[17] Geoffry Hinton. *Neural networks for machine learning*. Coursera, video lectures. Lecture 15b. 2012 (cit. on pp. 4, 9).

[18] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. “Deep residual learning for image recognition”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778 (cit. on pp. 1, 9, 11).

[19] Zhenliang He, Wangmeng Zuo, Meina Kan, Shiguang Shan, and Xilin Chen. “Attgan: Facial attribute editing by only changing what you want”. In: *IEEE Transactions on Image Processing* 28.11 (2019), pp. 5464–5478 (cit. on p. 19).

[20] Katherine Hermann and Andrew Lampinen. “What shapes feature representations? Exploring datasets, architectures, and training”. In: *Advances in Neural Information Processing Systems* 33 (2020) (cit. on p. 1).

[21] Gao Huang, Shichen Liu, Laurens Van der Maaten, and Kilian Q Weinberger. “Condensenet: An efficient densenet using learned group convolutions”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 2752–2761 (cit. on p. 4).

[22] Siyu Huang, Xi Li, Zhi-Qi Cheng, Zhongfei Zhang, and Alexander Hauptmann. “Gnas: A greedy neural architecture search method for multi-attribute learning”. In: *Proceedings of the 26th ACM international conference on Multimedia*. 2018, pp. 2049–2057 (cit. on pp. 11, 12).

[23] Sergey Ioffe and Christian Szegedy. “Batch normalization: Accelerating deep network training by reducing internal covariate shift”. In: *arXiv preprint arXiv:1502.03167* (2015) (cit. on p. 8).

[24]Naeem Ul Islam and Sukhan Lee. “Interpretation of deep CNN based on learning feature reconstruction with feedback weights”. In: *IEEE Access* 7 (2019), pp. 25195–25208 (cit. on p. 1).

[25]Eric Jang, Shixiang Gu, and Ben Poole. “Categorical reparameterization with gumbel-softmax”. In: *arXiv preprint arXiv:1611.01144* (2016) (cit. on p. 3).

[26]Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980* (2014) (cit. on p. 55).

[27]Nikolaus Kriegeskorte, Marieke Mur, and Peter A Bandettini. “Representational similarity analysis-connecting the branches of systems neuroscience”. In: *Frontiers in systems neuroscience* 2 (2008), p. 4 (cit. on p. 3).

[28]Alex Krizhevsky, Geoffrey Hinton, et al. “Learning multiple layers of features from tiny images”. In: (2009) (cit. on pp. 2, 31).

[29]Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. “Imagenet classification with deep convolutional neural networks”. In: *Communications of the ACM* 60.6 (2017), pp. 84–90 (cit. on pp. 1, 4, 12).

[30]C-C Jay Kuo, Min Zhang, Siyang Li, Jiali Duan, and Yueru Chen. “Interpretable convolutional neural networks via feedforward design”. In: *Journal of Visual Communication and Image Representation* 60 (2019), pp. 346–359 (cit. on p. 1).

[31]Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. “Pruning filters for efficient convnets”. In: *arXiv preprint arXiv:1608.08710* (2016) (cit. on p. 5).

[32]Mingwei Li, Zhenge Zhao, and Carlos Scheidegger. “Visualizing Neural Networks with the Grand Tour”. In: *Distill* (2020). https://distill.pub/2020/grand-tour (cit. on p. 6).

[33]Zhuang Liu, Jianguo Li, Zhiqiang Shen, et al. “Learning efficient convolutional networks through network slimming”. In: *Proceedings of the IEEE International Conference on Computer Vision*. 2017, pp. 2736–2744 (cit. on p. 5).

[34]Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. “Large-scale celebfaces attributes (celeba) dataset”. In: *Retrieved August* 15 (2018), p. 2018 (cit. on pp. 1, 11).

[35]Jian-Hao Luo and Jianxin Wu. “Autopruner: An end-to-end trainable filter pruning method for efficient deep model inference”. In: *Pattern Recognition* (2020), p. 107461 (cit. on p. 5).

[36]Andrew L Maas, Awni Y Hannun, and Andrew Y Ng. “Rectifier nonlinearities improve neural network acoustic models”. In: *Proc. icml*. Vol. 30. 1. 2013, p. 3 (cit. on p. 47).

[37]Sébastien Marcel and Yann Rodriguez. “Torchvision the machine-vision package of torch”. In: *Proceedings of the 18th ACM international conference on Multimedia*. 2010, pp. 1485–1488 (cit. on pp. 11, 31).

[38] Elliot Meyerson and Risto Miikkulainen. “Beyond shared hierarchies: Deep multitask learning through soft layer ordering”. In: *arXiv preprint arXiv:1711.00108* (2017) (cit. on p. 4).

[39] Richard Meyes, Melanie Lu, Constantin Waubert de Puiseau, and Tobias Meisen. “Ablation studies in artificial neural networks”. In: *arXiv preprint arXiv:1901.08644* (2019) (cit. on pp. 7, 16).

[40] Diganta Misra. “Mish: A self regularized non-monotonic neural activation function”. In: *arXiv preprint arXiv:1908.08681* (2019) (cit. on p. 47).

[41] Pavlo Molchanov, Stephen Tyree, Tero Karras, Timo Aila, and Jan Kautz. “Pruning convolutional neural networks for resource efficient inference”. In: *arXiv preprint arXiv:1611.06440* (2016) (cit. on p. 12).

[42] Grégoire Montavon, Wojciech Samek, and Klaus-Robert Müller. “Methods for interpreting and understanding deep neural networks”. In: *Digital Signal Processing* 73 (2018), pp. 1–15 (cit. on p. 33).

[43] Ari S Morcos, David GT Barrett, Neil C Rabinowitz, and Matthew Botvinick. “On the importance of single directions for generalization”. In: *arXiv preprint arXiv:1803.06959* (2018) (cit. on pp. 7, 36).

[44] Chris Olah, Nick Cammarata, Ludwig Schubert, et al. “Zoom In: An Introduction to Circuits”. In: *Distill* 5.3 (2020), e00024–001 (cit. on pp. 1, 6, 14).

[45] Chris Olah, Alexander Mordvintsev, and Ludwig Schubert. “Feature visualization”. In: *Distill* 2.11 (2017), e7 (cit. on pp. 6, 7, 13, 39, 48).

[46] Chris Olah, Arvind Satyanarayan, Ian Johnson, et al. “The building blocks of interpretability”. In: *Distill* 3.3 (2018), e10 (cit. on p. 6).

[47] Adam Paszke, Sam Gross, Francisco Massa, et al. “Pytorch: An imperative style, high-performance deep learning library”. In: *Advances in neural information processing systems*. 2019, pp. 8026–8037 (cit. on p. 11).

[48] Hieu Pham, Melody Y Guan, Barret Zoph, Quoc V Le, and Jeff Dean. “Efficient neural architecture search via parameter sharing”. In: *arXiv preprint arXiv:1802.03268* (2018) (cit. on p. 3).

[49] Boris T Polyak. “Some methods of speeding up the convergence of iteration methods”. In: *USSR Computational Mathematics and Mathematical Physics* 4.5 (1964), pp. 1–17 (cit. on p. 55).

[50] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V Le. “Regularized evolution for image classifier architecture search”. In: *Proceedings of the aaai conference on artificial intelligence*. Vol. 33. 2019, pp. 4780–4789 (cit. on p. 3).

[51] Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, et al. “Grad-cam: Visual explanations from deep networks via gradient-based localization”. In: *Proceedings of the IEEE international conference on computer vision*. 2017, pp. 618–626 (cit. on p. 6).

[52]Ozan Sener and Vladlen Koltun. “Multi-task learning as multi-objective optimization”. In: *Advances in Neural Information Processing Systems*. 2018, pp. 527–538 (cit. on pp. 11, 12).

[53]Albert Shaw, Daniel Hunter, Forrest Landola, and Sammy Sidhu. “Squeezenas: Fast neural architecture search for faster semantic segmentation”. In: *Proceedings of the IEEE international conference on computer vision workshops*. 2019, pp. 0–0 (cit. on p. 3).

[54]Dinggang Shen, Guorong Wu, and Heung-Il Suk. “Deep learning in medical image analysis”. In: *Annual review of biomedical engineering* 19 (2017), pp. 221–248 (cit. on p. 1).

[55]Mukund Sundararajan, Ankur Taly, and Qiqi Yan. “Axiomatic attribution for deep networks”. In: *Proceedings of the 34th International Conference on Machine Learning-Volume 70*. 2017, pp. 3319–3328 (cit. on p. 6).

[56]Mingxing Tan, Ruoming Pang, and Quoc V Le. “Efficientdet: Scalable and efficient object detection”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2020, pp. 10781–10790 (cit. on p. 1).

[57]Pang-Ning Tan, Michael Steinbach, and Vipin Kumar. *Introduction to data mining*. Pearson Education India, 2016 (cit. on p. 6).

[58]Andrew Tao, Karan Sapra, and Bryan Catanzaro. “Hierarchical Multi-Scale Attention for Semantic Segmentation”. In: *arXiv preprint arXiv:2005.10821* (2020) (cit. on p. 1).

[59]Simon Vandenhende, Stamatios Georgoulis, Bert De Brabandere, and Luc Van Gool. “Branched multi-task networks: deciding what layers to share”. In: *arXiv preprint arXiv:1904.02920* (2019) (cit. on p. 3).

[60]LN Vasershtein. “Markov processes on a countable product of spaces describing large automated systems”. In: *Probl. Inform. Trans* 14 (1969), pp. 64–73 (cit. on p. 26).

[61]Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017), pp. 5998–6008 (cit. on p. 48).

[62]Xijun Wang, Meina Kan, Shiguang Shan, and Xilin Chen. “Fully learnable group convolution for acceleration of deep neural networks”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2019, pp. 9049–9058 (cit. on p. 4).

[63]Po-Wei Wu, Yu-Jing Lin, Che-Han Chang, Edward Y Chang, and Shih-Wei Liao. “Relgan: Multi-domain image-to-image translation via relative attributes”. In: *Proceedings of the IEEE International Conference on Computer Vision*. 2019, pp. 5914–5922 (cit. on p. 19).

[64]*xdot*. `https://github.com/jrfonseca/xdot.py`. 2020 (cit. on p. 13).

[65]Qizhe Xie, Minh-Thang Luong, Eduard Hovy, and Quoc V Le. “Self-training with noisy student improves imagenet classification”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2020, pp. 10687–10698 (cit. on p. 1).

[66] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. “Aggregated residual transformations for deep neural networks”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 1492–1500 (cit. on pp. 3, 4).

[67] Jiecao Yu, Andrew Lukefahr, David Palframan, et al. “Scalpel: Customizing dnn pruning to the underlying hardware parallelism”. In: *ACM SIGARCH Computer Architecture News* 45.2 (2017), pp. 548–560 (cit. on p. 5).

[68] Ming Yuan and Yi Lin. “Model selection and estimation in regression with grouped variables”. In: *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 68.1 (2006), pp. 49–67 (cit. on p. 4).

[69] Quan-shi Zhang and Song-Chun Zhu. “Visual interpretability for deep learning: a survey”. In: *Frontiers of Information Technology & Electronic Engineering* 19.1 (2018), pp. 27–39 (cit. on p. 1).

[70] Quanshi Zhang, Ruiming Cao, Feng Shi, Ying Nian Wu, and Song-Chun Zhu. “Interpreting cnn knowledge via an explanatory graph”. In: *arXiv preprint arXiv:1708.01785* (2017) (cit. on p. 5).

[71] Zhaoyang Zhang, Jingyu Li, Wenqi Shao, et al. “Differentiable learning-to-group channels via groupable convolutional neural networks”. In: *Proceedings of the IEEE International Conference on Computer Vision*. 2019, pp. 3542–3551 (cit. on p. 4).

[72] Bolei Zhou, Yiyou Sun, David Bau, and Antonio Torralba. “Revisiting the importance of individual units in cnns via ablation”. In: *arXiv preprint arXiv:1806.02891* (2018) (cit. on pp. 1, 7, 35, 36).

[73] Michael Zhu and Suyog Gupta. “To prune, or not to prune: exploring the efficacy of pruning for model compression”. In: *arXiv preprint arXiv:1710.01878* (2017) (cit. on p. 5).

[74] Barret Zoph and Quoc V Le. “Neural architecture search with reinforcement learning”. In: *arXiv preprint arXiv:1611.01578* (2016) (cit. on p. 3).

Appendix A

A.1 CelebA hyperparameters

The networks were trained via Stochastic Gradient Descent with momentum [49], the learning rate was set to 0.01, momentum was set to 0.9, the batch size was set to 256. Images were resized to (64, 64) and standardized, no augmentations were used.

The connections were learned via Adam [26], the learning rate was set to 0.0005, L_1 regularization strength was set to 0.000003. The regularization was not applied for the first 5 epochs of the training. When training a sparse model, it was found that some unbalanced tasks are given up on by the network, with their balanced accuracy becoming equal to 0.5. Weights of losses for these tasks were increased from 0.025 to 0.1 (weights for other tasks were unchanged at 0.025). These tasks were: "5 o'Clock Shadow", "Bald", "Bangs", "Big Lips", "Blond Hair", "Blurry", "Goatee", "Gray Hair", "Narrow Eyes", "Pale Skin", "Pointy Nose", "Receding Hairline", "Rosy Cheeks", "Straight Hair", "Wearing Hat", "Wearing Necklace".

The networks were trained for 180 epochs with early stopping based on the average balanced accuracy on the validation set.

A.2 CIFAR-10 hyperparameters

The networks were trained via Stochastic Gradient Descent with momentum, the learning rate was set to 0.1, momentum was set to 0.9, the batch size was set to 128. Images were padded by 4 pixels and randomly cropped to (32, 32), then color jitter with parameters (brightness=0.05, contrast=0.05, saturation=0.05, hue=0.05) was applied. Afterwards, random affine transform was applied, with rotation by a degree from the $[0, 5]$ range, vertical and horizontal translation by no more than 10% of the image, scaling by a factor from the $[1.0, 1.2]$ range. Finally, the image was standardized.

The networks were trained for 120 epochs.